

Chapter 8

BC: A Calculator

8.1 Introduction 8-1

8.2 Demonstration 8-1

8.3 Tasks 8-3

8.3.1 Computing with Integers 8-4

8.3.2 Specifying Input and Output Bases 8-5

8.3.3 Scaling Quantities 8-6

8.3.4 Using Functions 8-8

8.3.5 Using Subscripted Variables 8-9

8.3.6 Using Control Statements: if, while and for 8-10

8.3.7 Using Other Language Features 8-12

8.4 Language Reference 8-14

8.4.1 Tokens 8-14

8.4.2 Expressions 8-15

8.4.3 Function Calls 8-16

8.4.4 Unary Operators 8-16

8.4.5 Multiplicative Operators 8-17

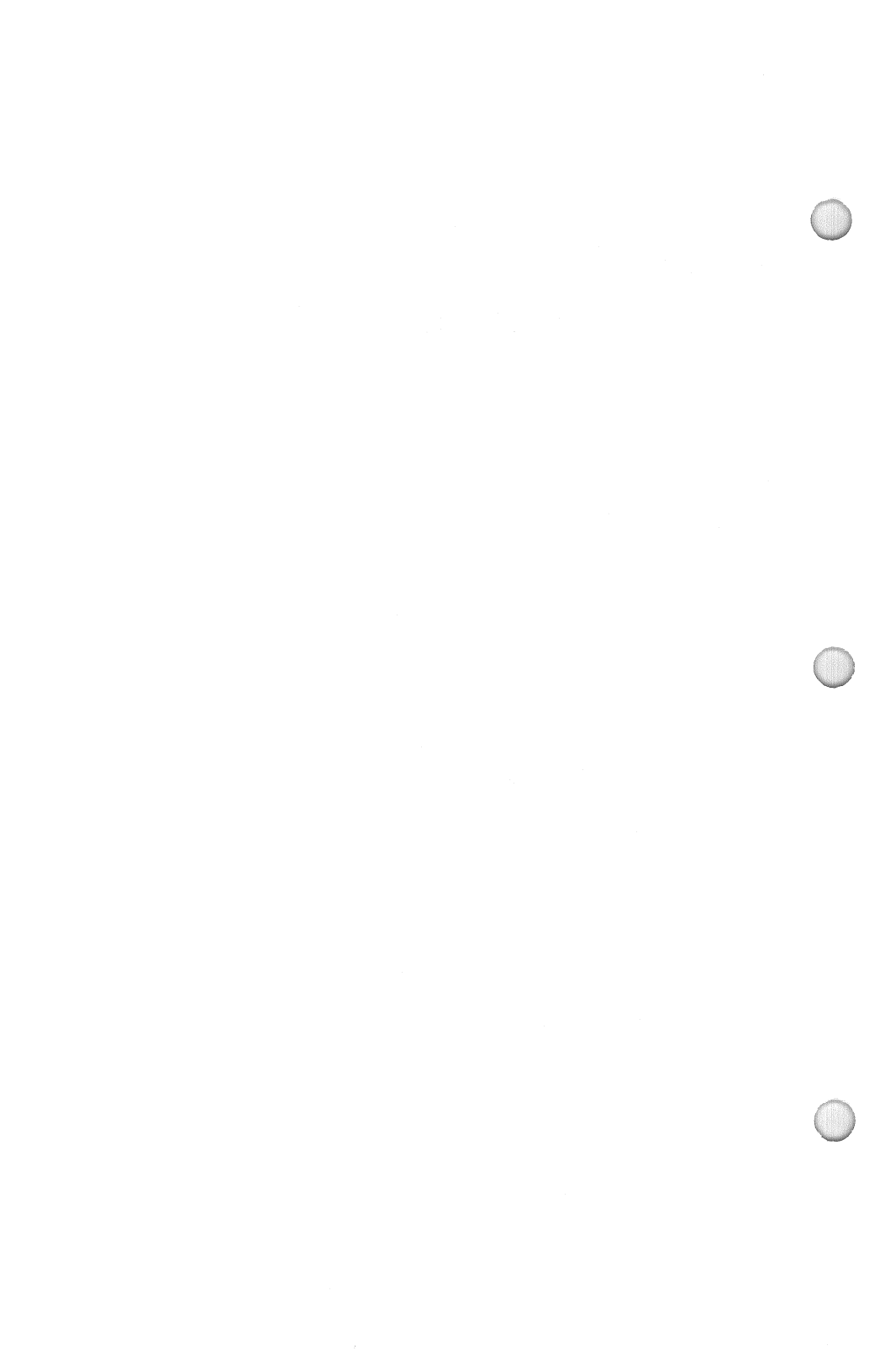
8.4.6 Additive Operators 8-17

8.4.7 Assignment Operators 8-18

8.4.8 Relational Operators 8-18

8.4.9 Storage Classes 8-18

8.4.10 Statements 8-19



8.1 Introduction

BC is a program that can be used as an arbitrary precision arithmetic calculator. BC's output is interpreted and executed by a collection of routines which can input, output, and do arithmetic on indefinitely large integers and on scaled fixed-point numbers. Although you can write substantial programs with BC, it is often used as an interactive tool for performing calculator-like computations. The language supports a complete set of control structures and functions that can be defined and saved for later execution. The syntax of BC has been deliberately selected to agree with the C language; those who are familiar with C will find few surprises. A small collection of library functions is also available, including sin, cos, arctan, log, exponential, and Bessel functions of integer order.

Common uses for BC are

- Computation with large integers
- Computations accurate to many decimal places
- Conversions of numbers from one base to another base

A scaling provision permits the use of decimal point notation. Provision is made for input and output in bases other than decimal. Numbers can be converted from decimal to octal simply by setting the output base equal to 8.

The actual limit on the number of digits that can be handled depends on the amount of storage available on the machine, so manipulation of numbers with many hundreds of digits is possible.

8.2 Demonstration

This demonstration is designed to show you

- How to get into and out of BC
- How to perform simple computations
- How expressions are formed and evaluated
- How to assign values to registers

To invoke BC, use the command

```
bc
```

To exit BC, type

quit

or press CTRL-D. Once you have entered BC, you can use it very much like a normal calculator. As with the XENIX shell, commands are read as command lines, so each line that you type must be terminated by a RETURN. Throughout this chapter, the RETURN is implied at the end of each command line. Within BC, normal processing of other keys, such as BKSP and INTERRUPT, also works.

For example, type the simple integer 5:

5

Output is immediately echoed on the next line to the standard output, which is normally the terminal screen:

5

Here “5” is a simple numeric expression. However, if you type the expression

5*5.25

(where the asterisk (*) is the multiplication operator) a computation is executed and the result printed on the next line:

26.25

What has happened here is that the line “5*5.25” has been evaluated, i.e., the expression has been reduced to its most elementary form, which is the number 26.25. The process of evaluation normally involves some type of computation such as multiplication, division, addition, or subtraction. For example, all four of these operations are involved in the following expression:

(10*5)+50-(50/2)

When this expression is evaluated, the subexpressions within parentheses are evaluated first, just as they would be with simple algebra, so that an intermediate step in the evaluation is “50+50-25” which ultimately reduces to the number “75”.

The simple addition

10.45+5.5555555

produces the output

16.0055555

Note how precision is retained in the above result.

The two-part multiplication

$(8*9)*7$

produces the answer

504

The last part of this demonstration shows you how to store values in special alphabetic registers. For example, type

$a=100; b=5$

What happens here is that the registers “a” and “b” are assigned the values 100 and 5, respectively. The semicolon is used here to place multiple BC statements on a single line, just as it is used in the XENIX shell. This command line produces no output because assignment statements are not considered expressions. However, the registers “a” and “b” can now be used in expressions. Thus you can now type

$a*b; a+b$

to produce

500
105

To exit BC, remember to type

quit

or press CTRL-D.

This ends the demonstration. Following sections describe use of BC in more detail. The final section of this chapter is a BC language reference.

8.3 Tasks

This section describes how to perform common BC tasks.

8.3.1 Computing with Integers

The simplest kind of statement is an arithmetic expression on a line by itself. For instance, if you type

142857 + 285714

and press RETURN, BC responds immediately with the line

428571

Other operators also can be used. The complete list includes

Operator	Represents
+	addition
-	subtraction
*	multiplication
/	devision
%	modulo (remaindering)
^	exponentiation

Division of integers produces an integer result truncated toward zero. Division by zero produces an error message.

Any term in an expression can be prefixed with a minus sign to indicate that it is to be negated (this is the “unary” minus sign). For example, the expression

7+-3

is interpreted to mean that -3 is to be added to 7.

More complex expressions with several operators and with parentheses are interpreted just as in FORTRAN, with exponentiation performed first, then multiplication, division, modulo, and finally, addition, and subtraction. The contents of parentheses are evaluated before expressions outside the parentheses. All of the operations are performed from left to right, except exponentiation, which is performed from right to left. Thus the following two expressions

$a^b c$ and $a^{(b^c)}$

are equivalent, as are the two expressions

$$a*b*c \text{ and } (a*b)*c$$

BC shares with FORTRAN and C the convention that $a/b*c$ is equivalent to $(a/b)*c$.

Internal storage registers to hold numbers have single lowercase letter names. The value of an expression can be assigned to a register in the usual way, thus the statement

$$x = x + 3$$

has the effect of increasing by 3 the value of the contents of the register named “x”. When, as in this case, the outermost operator is the assignment operator (=), then the assignment is performed but the result is not printed. There are 26 available named storage registers, one for each letter of the alphabet.

There is also a built-in square root function whose result is truncated to an integer (See also Section 8.3.3 for more information). For example, the lines

```
x = sqrt(191)
x
```

produce the printed result

13

8.3.2 Specifying Input and Output Bases

BC has special internal quantities called *ibase* and *obase*. *Ibase* is initially set to 10, and determines the base used for interpreting numbers that are read by BC. For example, the lines

```
ibase = 8
11
```

produce the output line

9

and you are all set up to do octal to decimal conversions. However, beware of trying to change the input base back to decimal by typing

```
ibase = 10
```

Because the number 10 is interpreted as octal, this statement has no effect.

For those who deal in hexadecimal notation, the characters A–F are permitted in numbers (no matter what base is in effect) and are interpreted as digits having values 10–15, respectively. These characters *must* be uppercase and not lowercase. The statement

```
ibase = A
```

changes you back to decimal input base no matter what the current input base is. Negative and large positive input bases are permitted; however no mechanism has been provided for the input of arbitrary numbers in bases less than 1 and greater than 16.

Cbase is used as the base for output numbers. The value of *obase* is initially set to a decimal 10. The lines

```
obase = 16
1000
```

produce the output line

```
3E8
```

This is interpreted as a three-digit hexadecimal number. Very large output bases are permitted. For example, large numbers can be output in groups of five digits by setting *obase* to 100000. Even strange output bases, such as negative bases, and 1 and 0, are handled correctly.

Very large numbers are split across lines with seventy characters per line. A split line that continues on the next line ends with a backslash (\). Decimal output conversion is fast, but output of very large numbers (i.e., more than 100 digits) with other bases is rather slow.

Remember that *ibase* and *obase* do not affect the course of internal computation or the evaluation of expressions; they affect only input and output conversion.

8.3.3 Scaling Quantities

A special internal quantity called *scale* is used to determine the scale of calculated quantities. Numbers can have up to 99 decimal digits after the decimal point. This fractional part is retained in further computations. We refer to the number of digits after the decimal point of a number as its *scale*.

When two scaled numbers are combined by means of one of the arithmetic operations, the result has a scale determined by the following rules:

Addition, subtraction

The scale of the result is the larger of the scales of the two operands. There is never

any truncation of the result.

Multiplication

The scale of the result is never less than the maximum of the two scales of the operands, never more than the sum of the scales of the operands, and subject to those two restrictions, the scale of the result is set equal to the contents of the internal quantity, *scale*.

Division

The scale of a quotient is the contents of the internal quantity, *scale*.

Modulo

The scale of a remainder is the sum of the scales of the quotient and the divisor.

Exponentiation

The result of an exponentiation is scaled as if the implied multiplications were performed. An exponent must be an integer.

Square Root

The scale of a square root is set to the maximum of the scale of the argument and the contents of *scale*.

All of the internal operations are actually carried out in terms of integers, with digits being discarded when necessary. In every case where digits are discarded, truncation is performed without rounding.

The contents of *scale* must be no greater than 99 and no less than 0. It is initially set to 0.

The internal quantities *scale*, *ibase*, and *base* can be used in expressions just like other variables. The line

$scale = scale + 1$

increases the value of *scale* by one, and the line

scale

causes the current value of *scale* to be printed.

The value of *scale* retains its meaning as a number of decimal digits to be retained in internal computation even when *ibase* or *obase* are not equal to 10. The internal computations (which are still conducted in decimal, regardless of the bases) are performed to the specified number of decimal digits, never hexadecimal or octal or any other kind of digits.

8.3.4 Using Functions

The name of a function is a single lowercase letter. Function names are permitted to use the same letters as simple variable names. Twenty-six different defined functions are permitted in addition to the twenty-six variable names. The line

```
define a(x){
```

begins the definition of a function with one argument. This line must be followed by one or more statements, which make up the body of the function, ending with a right brace (`}`). Return of control from a function occurs when a return statement is executed or when the end of the function is reached. The return statement can take either of the two forms

```
return
return(x)
```

In the first case, the returned value of the function is 0; in the second, it is the value of the expression in parentheses.

Variables used in functions can be declared as automatic by a statement of the form

```
auto x,y,z
```

There can be only one **auto** statement in a function and it must be the first statement in the definition. These automatic variables are allocated space and initialized to zero on entry to the function and thrown away on return. The values of any variables with the same names outside the function are not disturbed. Functions can be called recursively and the automatic variables at each call level are protected. The parameters named in a function definition are treated in the same way as the automatic variables of that function, with the single exception that they are given a value on entry to the function. An example of a function definition follows:

```
define a(x,y){
    auto z
    z = x*y
    return(z)
}
```

The value of this function, when called, will be the product of its two arguments.

A function is called by the appearance of its name, followed by a string of arguments enclosed in parentheses and separated by commas. The result is unpredictable if the wrong number of arguments are used.

If the function “a” is defined as shown in the previous example, then the line

```
a(7,3.14)
```

would print the result

```
21.98
```

Similarly, the line

```
x = a(a(3,4),5)
```

would cause the value of “x” to become 60.

Functions can require no arguments, but still perform some useful operation or return a useful result. Such functions are defined and called using parentheses with nothing between them. For example

```
b()
```

calls the function named *b*.

8.3.5 Using Subscripted Variables

A single lowercase letter variable name followed by an expression in brackets is called a *subscripted variable* and indicates an array element. The variable name is the name of the array and the expression in brackets is called the *subscript*. Only one-dimensional arrays are permitted in BC. The names of arrays are permitted to collide with the names of simple variables and function names. Any fractional part of a subscript is discarded before use. Subscripts must be greater than or equal to zero and less than or equal to 2047.

Subscripted variables can be freely used in expressions, in function calls and in return statements.

An array name can be used as an argument to a function, as in

```
f(a[ ])
```

Array names are automatic if used with empty brackets as a function parameter or in an auto declaration:

```
f(a[ ])
auto a[ ]
```

If the array is a function parameter, the entire contents of the array are copied for the use of the function, then thrown away on exit from the function. If the array is declared auto, the old contents of the array are not available to the

function. A new version of the array is initialized to zero when the function is entered and the old values are restored when the function is exited. Array names that refer to whole arrays cannot be used in any other context.

8.3.6 Using Control Statements: **if**, **while** and **for**

The **if**, **while**, and **for** statements are used to alter the flow within programs or to cause iteration. The range of each of these statements is a following statement or compound statement consisting of a collection of statements enclosed in braces. They are written as follows:

```

if ( relation ) statement
while ( relation ) statement
for ( expression1; relation; expression2 ) statement

if ( relation ) { statements }
while ( relation ) { statements }
for ( expression1; relation; expression2 ) { statements }

```

A relation in one of the control statements is an expression of the form

expression1 rel-op expression2

where the two expressions are related by one of the six relational operators:

<	less than
>	greater than
<=	less than/equal to
>=	greater than/equal to
==	equal to
!=	not equal to

Beware of using a single equal sign (=) instead of the double equal sign (==) in a relational. Both of these symbols are legal, so you will not get a diagnostic message. However, the operation will not perform the intended comparison.

The **if** statement causes execution of its range if and *only* if the relation is true. Then control passes to the next statement in the sequence.

The **while** statement causes repeated execution of its range, as long as the relation is true. The relation is tested before each execution of its range, and if the relation is false, control passes to the next statement beyond the range of the **while** statement.

The **for** statement begins by executing *expression1*. Then the relation is tested and, if true, the statements in the range of the **for** statement are executed. Then *expression2* is executed. The relation is tested, and so on. The typical use of the **for** statement is for a controlled iteration, as in the statement

```
for(i=1; i<=10; i=i+1) i
```

which will print the integers from 1 to 10.

The following are some examples of the use of the control statements:

```
define f(n){
    auto i, x
    x=1
    for(i=1; i<=n; i=i+1) x=x*i
    return(x)
}
```

The line

```
f(a)
```

prints “a” factorial if “a” is a positive integer.

The following is the definition of a function that computes values of the binomial coefficient (“m” and “n” are assumed to be positive integers):

```
define b(n,m){
    auto x, j
    x=1
    for(j=1; j<=m; j=j+1) x=x*(n-j+1)/j
    return(x)
}
```

The following function computes values of the exponential function by summing the appropriate series without regard to possible truncation errors:

```
scale = 20
define e(x){
    auto a, b, c, d, n
    a = 1
    b = 1
    c = 1
    d = 0
    n = 1
    while(1==1){
        a = a*x
        b = b*n
        c = c + a/b
        n = n + 1
        if(c==d) return(c)
        d = c
    }
}
```

8.3.7 Using Other Language Features

Every user should know the following language features:

- Statements are commonly typed one to a line. You can type several statements on a line if they are separated by semicolons.
- If an assignment statement is placed in parentheses, it then has a value and can be used anywhere that an expression can. For example, the line

```
(x=y+17)
```

not only makes the indicated assignment, but also prints the resulting value.

The following is an example of a use of the value of an assignment statement even when it is not placed in parentheses:

```
x = a[i=i+1]
```

This causes a value to be assigned to “x” and also increments “i”

before it is used as a subscript.

- Following are the BC constructions and their equivalents:

Construction	Equivalent
$x=y=z$	$x=(y=z)$
$x=+y$	$x=x+y$
$x=-y$	$x=x-y$
$x=*y$	$x=x*y$
$x=/y$	$x=x/y$
$x=\%y$	$x=x\%y$
$x=\^y$	$x=x^y$
$x++$	$(x=x+1)-1$
$x--$	$(x=x-1)+1$
$++x$	$x=x+1$
$--x$	$x=x-1$

If you type one of these constructions inadvertently, something legal but unexpected may happen. Be aware that in some of these constructions spaces are significant. There is a real difference between “ $x=-y$ ” and “ $x= -y$ ”. The first replaces “ x ” by “ $x-y$ ” and the second by “ $-y$ ”.

- The comment convention is identical to the C comment convention. Comments begin with “ $/*$ ” and end with “ $*/$ ”.
- You can obtain a library of math functions when you invoke BC by typing

bc -l

This command loads the library functions sine, cosine, arctangent, natural logarithm, exponential, and Bessel functions of integer order. These are named “ s ”, “ c ”, “ a ”, “ l ”, “ e ”, and “ $j(n,x)$ ”, respectively. This library sets *scale* to 20 by default.

- If you type

bc file ...

BC will read and execute the named file or files before accepting commands from the keyboard. In this way, you can load your own programs and function definitions.

8.4 Language Reference

This section is a comprehensive reference to the BC language. It contains a more concise description of the features mentioned in earlier sections.

8.4.1 Tokens

Tokens are keywords, identifiers, constants, operators, and separators. Token separators can be spaces, tabs or comments. Newline characters or semicolons separate statements.

Comments Comments are introduced by the characters “/*” and are terminated by “*/”.

Identifiers There are three kinds of identifiers: ordinary identifiers, array identifiers and function identifiers. All three types consist of single lowercase letters:

Ordinary identifiers are named expressions which represent a place where the current value of the expression is stored. Ordinary identifiers have no dependencies or arguments.

Array identifiers are followed by square brackets, enclosing an optional expression describing a subscript. Arrays are singly dimensioned and can contain up to 2048 elements. Indexing begins at 0, so an array can be indexed from 0 to 2047. Subscripts are truncated to integers.

Function identifiers are followed by parentheses, enclosing optional arguments.

The three types of identifiers do not conflict; a program can have a variable named “x”, an array named “x”, and a function named “x”, all of which are separate and distinct.

Keywords The following are reserved keywords:

auto	obase
break	return
define	scale
for	sqrt
ibase	quit
if	while
length	

Constants Constants are arbitrarily long numbers with an optional decimal point. The hexadecimal digits A-F are also recognized as digits with decimal values 10–15, respectively.

8.4.2 Expressions

All expressions can be evaluated to a value. The value of an expression is always printed unless the main operator is an assignment. The precedence of expressions (i.e., the order in which they are evaluated) is as follows:

Function calls
 Unary operators
 Multiplicative operators
 Additive operators
 Assignment operators
 Relational operators

There are several types of expressions:

Named expressions	Places where values are stored. Simply stated, named expressions are legal on the left side of an assignment. The value of a named expression is the value stored in the place named. • <i>Ordinary identifiers</i> are named expressions. They have an initial value of zero. • <i>Array elements</i> are named expressions. They have an initial value of zero. • The internal registers <i>scale</i>, <i>ibase</i>, and <i>obase</i> are all named expressions. <i>Scale</i> is the number of digits after the decimal point to be retained in arithmetic operations and has an initial value of zero. <i>Ibase</i> and <i>obase</i> are the input and output number radices respectively. Both <i>ibase</i> and <i>obase</i> have initial values of 10.
-------------------	--

Constants	Primitive expressions that evaluate to themselves.
-----------	--

Parenthetic Expressions	An expression surrounded by parentheses is
-------------------------	--

a primitive expression. The parentheses are used to alter normal operator precedence.

Function Calls

Expressions that return values. They are discussed in Section 8.4.3.

8.4.3 Function Calls

A function call consists of a function name followed by parentheses containing a comma-separated list of expressions, which are the function arguments. The syntax is as follows:

function-name ([*expression* [, *expression* ...]])

A whole array passed as an argument is specified by the array name followed by empty square brackets. All function arguments are passed by value. As a result, changes made to the formal parameters have no effect on the actual arguments. If the function terminates by executing a return statement, the value of the function is the value of the expression in the parentheses of the return statement, or 0 if no expression is provided or if there is no return statement. There are three built-in functions:

sqrt (<i>expr</i>)	The result is the square root of the expression. It is truncated in the least significant decimal place. The scale of the result is the scale of the expression or the value of <i>scale</i> , whichever is larger.
length (<i>expr</i>)	The result is the total number of significant decimal digits in the expression. The scale of the result is zero.
scale (<i>expr</i>)	The result is the scale of the expression. The scale of the result is zero.

8.4.4 Unary Operators

The unary operators bind right to left.

- <i>expr</i>	The result is the negative of the expression.
++ <i>named_expr</i>	The named expression is incremented by one. The result is the value of the named expression after incrementing.
-- <i>named_expr</i>	The named expression is decremented by one. The result is the value of the named expression after decrementing.

<i>named_expr++</i>	The named expression is incremented by one. The result is the value of the named expression before incrementing.
<i>named_expr--</i>	The named expression is decremented by one. The result is the value of the named expression before decrementing.

8.4.5 Multiplicative Operators

The multiplicative operators (*, /, and %) bind from left to right.

<i>expr*expr</i>	The result is the product of the two expressions. If “a” and “b” are the scales of the two expressions, then the scale of the result is $\min(a+b, \max(\text{scale}, a, b))$
<i>expr/expr</i>	The result is the quotient of the two expressions. The scale of the result is the value of <i>scale</i> .
<i>expr%expr</i>	The modulo operator (%) produces the remainder of the division of the two expressions. More precisely, $a\%b$ is $a-a/b*b$. The scale of the result is the sum of the scale of the divisor and the value of <i>scale</i> where <i>scale</i> is non-zero.
<i>expr^expr</i>	The exponentiation operator binds right to left. The result is the first expression raised to the power of the second expression. The second expression must be an integer. If “a” is the scale of the left expression and “b” is the absolute value of the right expression, then the scale of the result is $\min(a*b, \max(\text{scale}, a))$

8.4.6 Additive Operators

The additive operators bind left to right.

<i>expr+expr</i>	The result is the sum of the two expressions. The scale of the result is the maximum of the scales of the expressions.
<i>expr-exp</i>	The result is the difference of the two expressions. The scale of the result is the maximum of the scales of the expressions.

8.4.7 Assignment Operators

The following assignment operators assign values to the named expression on the left side:

<i>named_expr=expr</i>	Assigns the value of the expression on the right to the named expression on the left.	
<i>named_expr+=expr</i>	Result equivalent <i>named_expr=named_expr+expr.</i>	to
<i>named_expr-=expr</i>	Result equivalent <i>named_expr=named_expr-expr.</i>	to
<i>named_expr*=expr</i>	Result equivalent <i>named_expr=named_expr*expr.</i>	to
<i>named_expr/=expr</i>	Result equivalent <i>named_expr=named_expr/expr.</i>	to
<i>named_expr%=expr</i>	Result equivalent <i>named_expr=named_expr%expr.</i>	to
<i>named_expr^=expr</i>	Result equivalent <i>named_expr=named_expr^expr.</i>	to

8.4.8 Relational Operators

Unlike all other operators, the relational operators are valid only as the object of an **if** or **while** statement, or inside a **for** statement. The relational operators are

expr<expr
expr>expr
expr<=expr
expr>=expr
expr==expr
expr!=expr

8.4.9 Storage Classes

There are only two storage classes in BC: *global* and *automatic* (local). Only identifiers that are to be local to a function need be declared with the **auto**

command. The arguments to a function are local to the function. All other identifiers are assumed to be global and available to all functions.

All identifiers, global and local, have initial values of zero. Identifiers declared as **auto** are allocated on entry to the function and released on returning from the function: they do not retain values between function calls. Note that **auto** arrays are specified by the array namer, followed by empty square brackets.

Automatic variables in BC do not work the way they do in C. On entry to a function, the old values of the names that appear as parameters and as automatic variables are pushed onto a stack. Until return is made from the function, reference to these names refers only to the new values.

8.4.10 Statements

Statements must be separated by a semicolon or a newline. Except where altered by control statements, execution is sequential. There are four types of statements: *expression statements*, *compound statements*, *quoted string statements*, and *built-in statements*:

Expression statements	When a statement is an expression, unless the main operator is an assignment, the value of the expression is printed, followed by a newline character.
Compound statements	Statements grouped together and surrounded with curly braces ({ and }).
Quoted string statements	For example "string" prints the string inside the quotation marks.
Built-in statements	Built-in statements include auto , break , define , for , if , quit , return , and while .

- The **auto** statement causes the values of the identifiers to be pushed down. The identifiers can be ordinary identifiers or array identifiers. Array identifiers are specified by following the array name by empty square brackets. The **auto** statement must be the first statement in a function definition. Syntax of the **auto** statement is

`auto identifier [, identifier]`

- The **break** statement terminates a **for** or **while** statement.

Syntax for the break statement is

break

- The **define** statement defines a function; parameters to the function can be ordinary identifiers or array names. Array names must be followed by empty square brackets. The syntax of the define statement is

```
define ([parameter [ , parameter ... ]]) { statements }
```

- The **for** statement is the same as

```
first-expression  
while(relation) {  
    statement  
    last-expression  
}
```

All three expressions must be present. Syntax of the **for** statement is

```
for (expression; relation; expression) statement
```

- The **if** statement is executed if the relation is true. The syntax is

```
if (relation) statement
```

- The **quit** statement stops execution of a BC program and returns control to XENIX when it is first encountered. Because it is not treated as an executable statement, it cannot be used in a function definition or in an **if**, **for**, or **while** statement. Note that entering a CTRL-D at the keyboard is the same as typing "quit". The syntax of the quit statement is

quit

- The **return** statement terminates a function, pops its auto variables off the stack, and specifies the result of the function. The result of the function is the result of the expression in parentheses. The first form is equivalent to "return(0)". The syntax of the **return** statement is

```
return(expr)
```

- The **while** statement is executed while the relation is true. The test occurs before each execution of the statement. The syntax of the **while** statement is

while (*relation*) *statement*

Appendix A

Ed

A.1 Introduction A-1

A.2 Demonstration A-1

A.3 Basic Concepts A-2

A.3.1 The Editing Buffer A-2

A.3.2 Commands A-2

A.3.3 Line Numbers A-2

A.4 Tasks A-3

A.4.1 Entering the Editor A-3

A.4.2 Appending Text: a A-3

A.4.3 Writing Out a File: w A-3

A.4.4 Leaving The Editor: q A-4

A.4.5 Editing a New File: e A-4

A.4.6 Changing the File Name Written Out: f A-5

A.4.7 Reading in a File: r A-5

A.4.8 Displaying Lines on the Screen: p A-5

A.4.9 Displaying The Current Line: dot(.) A-7

A.4.10 Deleting Lines: d A-9

A.4.11 Performing Text Substitutions: s A-9

A.4.12 Searching A-11

A.4.13 Changing and Inserting Text: c and i A-15

A.4.14 Moving Lines: m A-15

A.4.15 Performing Global Commands: g and v A-17

A.4.16 Displaying Tabs and Control Characters: l A-19

A.4.17 Undoing Commands: u A-20

A.4.18 Marking Your Spot in a File: k A-20

A.4.19 Transferring Lines: t A-21

A.4.20 Escaping to the Shell: ! A-21

A.5 Context and Regular Expressions A-22

- A.5.1 Period: (.) A-23
- A.5.2 Backslash: \ A-25
- A.5.3 Dollar Sign: \$ A-26
- A.5.4 Caret: ^ A-28
- A.5.5 Asterisk: * A-28
- A.5.6 Brackets: [and] A-31
- A.5.7 Ampersand: & A-32
- A.5.8 Substituting New Lines A-33
- A.5.9 Joining Lines A-34
- A.5.10 Rearranging a Line: \ (and \) A-34
- A.6 Speeding Up Editing A-35
 - A.6.1 Semicolon: ; A-38
 - A.6.2 Interrupting the Editor A-39
- A.7 Cutting and Pasting with the Editor A-39
 - A.7.1 Inserting One File into Another A-40
 - A.7.2 Writing Out Part of a File A-40
- A.8 Editing Scripts A-41
- A.9 Summary of Commands A-42

A.1 Introduction

Ed is a text editor used to create and modify text. The text is normally a document, a program, or data for a program. Note that the line editor `ex`, available with other XENIX packages, is very similar to `ed`; you can use this chapter as an introduction to `ex` as well as to `ed`.

A.2 Demonstration

This section leads you through a simple session with `ed`, giving you a feel for how it is used and how it works. To begin the demonstration, invoke `ed` by typing

```
ed
```

This invokes the editor and begins your editing session. You are now at the command level of `ed`. To obtain a visual command prompt, type “P” and press RETURN. Initially, you are editing a temporary file called the *editing buffer*.

Typically, the first thing you will want to do with an empty buffer is add text to it. For example, after the prompt, type

```
a
this is line 1
this is line 2
this is line 3
this is line 4
CTRL-D
```

This *appends* four lines of text to the buffer. To view these lines on your screen, type

```
1,4p
```

where the “1,4” specifies a line number range and the `p` command “prints” the specified lines on the screen.

Now type

```
2p
```

to view line number two. Then type

```
p
```

This prints out the current line on the screen, which happens to be line number two. By default, most `ed` commands operate on only the current line.

A.3 Basic Concepts

This section illustrates some of **ed**'s basic concepts.

A.3.1 The Editing Buffer

Each time you invoke **ed**, XENIX allocates an area of computer memory in which you will perform all of your editing operations. This area is called the *editing buffer*. XENIX places a copy of the file you specify into the buffer for editing. Only when you *write out* your file (see Section A.4.3) do you affect the original file.

A.3.2 Commands

You enter commands by typing them at your keyboard and pressing RETURN. (Although our command examples do not include RETURN, its presence is assumed.) Most commands are single characters that can be preceded by the specification of a line number or a line number range. By default, most commands operate on the current line, described in Section A.3.3. Many commands take filename or string arguments that are used by the command when it is executed.

If you make an error while entering a command, **ed** will display a question mark. If you want **ed** to display informative messages, type H and press RETURN. Now, **ed** will display the messages in the format

```
?  
error message
```

A.3.3 Line Numbers

Whenever you execute a command that changes the number of lines in the editing buffer, **ed** immediately renumbers the lines. Many editing commands will take either single line numbers or line number ranges as prefixing arguments. These arguments specify the actual lines in the editing buffer that are to be affected by the given command. By default, a special line number called *dot* specifies the current line. See Section A.4.9 for more information on dot.

A.4 Tasks

This section discusses the tasks you perform in everyday editing.

A.4.1 Entering the Editor

To enter **ed**, type

```
ed filename
```

where *filename* is the name of a new or existing file.

A.4.2 Appending Text: a

The **append** command lets you add text to the buffer. To use **append**, type

```
a
```

followed by the desired text. To stop appending, type a period on a line by itself. For example

```
a
Now is the time
for all good men
to come to the aid of their party.
```

The period (.) tells **ed** that you have finished appending. (You can also use CTRL-D, but we will use the period throughout this discussion.) If **ed** doesn't respond, type an extra line with just a period (.) on it.

After appending is complete, the buffer contains the following three lines:

```
Now is the time
for all good men
to come to the aid of their party.
```

The “a” and period don't appear, because they are not text.

A.4.3 Writing Out a File: w

The **write** command lets you write out the contents of the editing buffer into a file, overwriting the file's previous contents. For example, to save the text in a file named *text*, type

```
w text
```

Ed responds by printing the number of characters it has written out. (Spaces

and the newline character at the end of each line are included in the character count.) Writing out a file just makes a copy of the text—the buffer's contents are not disturbed, so you can go on adding text to it. If you invoked **ed** with the command "**ed filename**", then by default a **w** command by itself will write the buffer out to *filename*.

No change in the contents of a file takes place until you give a **w** command. It's a good idea to write out the text to a file from time to time as you create it. If the system crashes, you will lose all the text in the buffer, but still have access to any text that was written out to a file.

A.4.4 Leaving The Editor: **q**

To terminate an **ed** session, save the text you're working on by writing it to a file using the **w** command, then type

q

The system responds with the XENIX prompt character. If you try to quit without writing out the file **ed** will print

?

At that point, write out the text if you want to save it; if not, typing another "**q**" will get you out of the editor.

A.4.5 Editing a New File: **e**

To read existing text into your editing buffer, use the **edit** command, which places the entire contents of a file in the buffer. For example, the **edit** command lets you edit text you have previously saved with the **w** command.

If you had saved the three lines "Now is the time", etc., with a **w** command in an earlier session, the command

e text

would place the entire contents of the file *text* into the buffer and respond with

68

which is the number of characters in *text*. If anything is already in the buffer, it is deleted first.

If you use the **e** command to read a file into the buffer, you don't need to use a filename after a subsequent **w** command. **Ed** remembers the last filename used in an **e** command, and **w** will write to this file. Thus, a good way to

operate is this:

```
ed
e file
[editing session]
w
q
```

This way, you can type **w** from time to time and be secure in the knowledge that if you typed the filename right in the beginning, you are writing out to the proper file each time.

A.4.6 Changing the File Name Written Out: **f**

To find out the last file written to, use the **file(f)** command. Just type **f** without a filename. You can also change the name of the remembered filename with **f**. Thus a useful sequence is

```
ed precious
f junk
```

which gets a copy of the file named *precious*, then uses **f** to save the text in the file *junk*. The original file will be preserved as *precious*.

A.4.7 Reading in a File: **r**

To read a file into the buffer without destroying what is already there, use the **read(r)** command. The command

```
r text
```

reads the file *text* into your editing buffer and adds it to the end of whatever is already in the buffer. This function is useful for combining files.

Like the **w** and **e** commands, after the reading operation is complete, **r** prints the number of characters read in.

A.4.8 Displaying Lines on the Screen: **p**

Use the **print(p)** command to print the contents of the editing buffer (or parts of it) on the terminal screen. Specify the lines where you want printing to begin and where you want it to end, separated by a comma and followed by the letter “**p**”. Thus, to print the first two lines of the buffer (that is, lines 1 through 2), type

```
1,2p
```

Ed responds with

```
Now is the time
for all good men
```

To print *all* the lines in the buffer, use the shorthand symbol — the dollar sign (\$) to indicate the last line in the buffer:

```
1,$p
```

This will print *all* the lines in the buffer (from line 1 to the last line). To stop printing before it is finished, press the INTERRUPT key. Ed then displays

```
?
interrupt
```

and waits for the next command.

To print the *last* line of the buffer, type

```
$p
```

To print a single line, type *just* the line number. If you type

```
$
```

ed prints the last line of the buffer.

You can also use \$ in combinations like

```
$-1,$p
```

which prints the last two lines of the buffer. This helps when you want to see how far you are in your typing.

The next step is to use address arithmetic to combine the line numbers like dot (.) and dollar sign (\$) with plus (+) and minus (-). (Note that “dot” is shorthand for the current line, and is discussed in Section A.4.9) Thus

```
$-1
```

prints the next to last line of the current file (that is, one line before the line \$). For example, to recall how far you were in a previous editing session

```
$-5,$p
```

prints the last six lines. (Be sure you understand why it's six, not five.) If there aren't six lines in the file, you'll get an error message.

The command

```
.-3, +3p
```

prints from three lines before the current line (line dot) to three lines after. The plus (+) can be omitted:

```
.-3,.3p
```

is identical in meaning.

Another area in which you can save typing effort in specifying lines is to use plus and minus as line numbers by themselves. For example

```
-
```

by itself is a command to move back one line in the file. In fact, you can string several minus signs together to move back that many lines. For example

```
---
```

moves back three lines, as does

```
-3
```

A.4.9 Displaying The Current Line: dot (.)

Suppose your editing buffer contains the following six lines:

```
Now is the time
for all good men
to come to the aid of their party.
Now is the time
for all good men
to come to the aid of their party.
```

If you type

```
1,3p
```

ed displays

```
Now is the time
for all good men
to come to the aid of their party.
```

Try typing

p

This prints

```
to come to the aid of their party.
```

which is the third line of the buffer. In fact, it is the last (most recent) line that you have done anything with. You can repeat this **p** command without line numbers, and **ed** will continue to print line 3.

This happens because **ed** maintains a record of the last line that you did anything to (in this case, line 3, which you just printed). The line most recently acted on is referred to with a period (.) and is called *dot*. Dot is a line number in the same way that dollar (\$) is; it means “the current line”, or loosely, “the line you most recently did something to.” You can use it in several ways. One possibility is to type

```
.,$p
```

This will print all the lines from (and including) the current line clear to the end of the buffer. In our example these are lines 3 through 6.

Some commands change the value of dot, while others do not. The **p** command sets dot to the number of the last line printed. In the example above, **p** sets dot to 6.

Dot is often used in combinations like this one:

```
.,+1
```

Or equivalently:

```
.,+1p
```

This means “print the next line” and is one way of stepping slowly through the editing buffer. You can also type

```
.,-1
```

This means “print the line *before* the current line.” This enables you to go backwards through the file if you wish. Another useful command is something like

```
.,-3,-1p
```

which prints the previous three lines.

Don't forget that all of these change the value of dot. To print the value of dot,

type

```
.=
```

Essentially, **p** can be preceded by zero, one, or two line numbers. If no line number is given, **ed** prints the current line, the line that dot refers to. If one line number is given (with or without the letter **p**), **ed** prints that line (and dot is set there); and if two line numbers are given, **ed** prints all the lines in that range (and sets dot to the last line printed). If two line numbers are specified, the first cannot be bigger than the second.

Pressing RETURN once causes printing of the next line. It is equivalent to

```
+.1p
```

Next, try typing a minus sign (–) by itself; it is equivalent to typing

```
-.1p
```

A.4.10 Deleting Lines: **d**

You use the **delete(d)** command to delete lines in the buffer. The lines to be deleted are specified for **d** exactly as they are for **p**. Thus, the command

```
4,$d
```

deletes lines 4 through the end. There are now three lines left in our example, as you can check by typing:

```
1,$p
```

Notice that **\$** now is line 3. Dot is set to the line after the last line deleted, unless the last line deleted is the last line in the buffer. In that case, dot is set to **\$**.

A.4.11 Performing Text Substitutions: **s**

The **substitute(s)** command lets you change individual words or letters within a line or group of lines. It is the command you use to correct spelling mistakes and typing errors.

Suppose that, due to a typing error, line 1 says

```
Now is th time
```

The letter “e” has been left off of the word “the”. You can use **s** to correct this:

```
1s/th/the/
```

This substitutes the characters “the” for the characters “th” in line 1. To verify that the substitution has worked, type

```
p
```

```
to get
```

```
Now is the time
```

which is what you wanted. Notice that dot must be the line where the substitution took place, since the **p** command printed that line. Dot is always set this way with the **s** command.

The syntax for the substitute command is

```
[starting-line,ending-line]s/pattern/replacement/cmds
```

The string of characters between the first pair of slashes is replaced by whatever is between the second pair, in *all* the lines between *starting-line* and *ending-line*. Only the first occurrence on each line is changed, however. Changing *every* occurrence is discussed later in this section. The rules for line numbers are the same as those for **p**, except that dot is set to the last line changed. If no substitution takes place, dot is *not* changed, resulting in the error message

```
?  
search string not found
```

Thus, you can type

```
1,$s/speling/spelling/
```

and correct the first spelling mistake on each line in the text.

To change all occurrences of the text sequence, add a **g** (for “global”) to the **s** command:

```
s/.../.../g
```

If no line numbers are given, the **s** command assumes we mean “make the substitution on line dot”, so it changes things only on the current line. This leads to the very common sequence

```
s/something/something else/p
```

which makes a correction on the current line, then prints it to make sure the

correction worked out right. If it didn't, you can try again. (Notice that the **p** is on the same line as the **s** command. With few exceptions, **p** can follow any command; no other multicommand lines are legal.)

It is also legal to type

```
s/string//
```

which means “change the first string of characters to nothing” or, in other words, remove them. This is useful for deleting extra words in a line or removing extra letters from words. For instance, if you had

```
Nowxx is the time
```

you could type

```
s/xx//p
```

to get

```
Now is the time
```

Notice that two adjacent slashes mean “no characters”, not a space. There *is* a difference.

A.4.12 Searching

Now that you've mastered the substitute command, you can move on to mastering another important concept: context searching.

Suppose you have the original three-line text in the buffer:

```
Now is the time
for all good men
to come to the aid of their party.
```

Suppose you want to find the line that contains the word “their”, so that you can change it to the word “the”. With only three lines in the buffer, it's pretty easy to keep track of which line the word “their” is on. But if the buffer contained several hundred lines, and you'd been making changes, deleting and rearranging lines, and so on, you would no longer really know what this line number would be. Context searching is simply a method of specifying the desired line, regardless of its number, by specifying a textual pattern contained in the line.

The way to say search for a line that contains this particular string of characters is to type

/search string/

For example, the **ed** command

/their/

locates the next occurrence of the characters between the slashes (i.e., “their”). Note that you do not need to type the final slash. The above search command is the same as typing

/their

The search command sets dot to the line on which the pattern is found and prints it for verification:

to come to the aid of their party.

“Next occurrence” means that **ed** starts looking for the string at line “+.1”, searches to the end of the buffer, then continues at line 1 and searches to line dot. (That is, the search “wraps around” from \$ to 1.) It scans all the lines in the buffer until it either finds the desired line or returns to dot. If the given string of characters can’t be found in any line, **ed** prints the error message

?
search string not found

Otherwise, **ed** prints the line it found.

You can also search *backwards* in a file for search strings by using question marks instead of slashes. For example

?thing?

searches backwards in the file for the word “thing” as does

?thing

The slash and question mark are the only characters you can use to delimit a search, though you can use any character in a substitute command. If you get unexpected results using any of the characters

*^ . \$ [* \ &*

read Section A.5.

You can search for the desired line *and* make a substitution at the same time by using a command of the form

/their/s/their/the/p

This yields:

to come to the aid of the party.

The above command contains three separate actions:

- a context search for the desired line
- a substitution
- the printing of the line

The expression “/their/” is a context search expression. In their simplest form, all context search expressions are like this—a string of characters surrounded by delimiters. Context searches are interchangeable with line numbers, so they can be used by themselves to find and print a desired line, or as line numbers for some other command, like s. They were used both ways in the previous examples.

Suppose the buffer contains the three familiar lines

Now is the time
for all good men
to come to the aid of their party.

The ed line numbers

/Now/+1
/good/
/party/-1

are all context search expressions, and they all refer to the same line (line 2). To make a change in line 2, you could type

/Now/+1s/good/bad/

or

/good/s/good/bad/

or

/party/-1s/good/bad/

The choice is dictated only by convenience. For instance, you could print all three lines by typing

```
/Ncw/,/party/p
```

or

```
/Ncw/,/Now/+2p
```

or any similar combination. The first combination is better if you don't know how many lines are involved.

The basic rule is that a context search expression is the same as a line number, so it can be used wherever a line number is needed.

Suppose you search for

```
/horrible thing/
```

and when the line is printed you discover that it isn't the "horrible thing" that you wanted, so it is necessary to repeat the search. You don't have to retype the search: just type

```
//
```

This is a shorthand expression for "the previous thing that was searched for", whatever it was. This can be repeated as many times as necessary. You can also go backwards, since

```
??
```

searches for the same thing, but in the reverse direction.

You can also use // as the left side of a substitute command, to mean "the most recent pattern." For example, if you've just found the string

```
/horrible thing/
```

you can change it by typing

```
s//good/p
```

This changes "horrible thing" to "good". To change previous occurrences of "horrible thing" to "good", type

```
??s//good/
```

A.4.13 Changing and Inserting Text: **c** and **i**

This section discusses the **change(c)** command, which is used to change or replace one or more lines, and the **insert(i)** command, which is used for inserting one or more lines.

The **c** command is used to replace a number of lines with different lines that you type at the terminal. For example, to change lines “+.1” through “\$” to something else, type

```
.+1,$c
type the lines of text you want here ...
.
```

The lines you type between the **c** command and the dot (.) will replace the originally addressed lines. This is useful in replacing a line or several lines that have errors in them.

If only one line is specified in the **c** command, then only that line is replaced. (You can type in as many replacement lines as you like.) Notice the use of a period to end the input. This works just like the period in the append command and must appear by itself on a new line. If no line number is given, the current line specified by dot is replaced. The value of dot is set to the last line you typed in. Note that the line referenced by dot and the terminating period are completely different: the first is used simply to terminate a command, the second points at a specific line of text.

The **i** command is similar to the append command. For example

```
/search string/i
type the lines to be inserted here ...
.
```

inserts the given text *before* the next line that contains “search string”. The text between **i** and the terminating period is *inserted before* the specified line. If no line number is specified, dot is used. Dot is set to the last line inserted.

A.4.14 Moving Lines: **m**

The **move(m)** command lets you move a group of lines from one place to another in the buffer. Suppose you want to move the first three lines of the buffer to the end of the buffer. You *could* do it by typing

```
1,3w temp
$r temp
1,3d
```

where *temp* is the name of a temporary file. However, you can do it more easi-

ly with the **m** command:

```
1,3m$
```

This will move lines 1 through 3 to the end of the file.

To move text, use a command of the form

```
start-line,end-linemafter-this-line
```

The third line specified is the place where the moved text is placed. Of course, the lines to be moved can be specified by context searches. If you had

```
First paragraph
end of first paragraph.
Second paragraph
end of second paragraph.
```

you could reverse the two paragraphs like this:

```
/Second/,/end of second/m/First/-1
```

Notice the -1. The moved text goes *after* the line mentioned. Dot gets set to the last line moved. Your file will now look like this:

```
Second paragraph
end of second paragraph
First paragraph
end of first paragraph
```

As another example of a frequent operation, you can reverse the order of two adjacent lines by moving the first line after the second line. Suppose that you are positioned at the first line. Then

```
m+
```

moves line dot to one line after the current line dot. If you are positioned on the second line,

```
m- -
```

moves line dot to one line before the current line dot.

The **m** command is more succinct than writing, deleting and rereading. The main difficulty with the **m** command is that if you use patterns to specify both the lines you are moving and the target, you must specify them properly, or you won't move the lines you want. The result of a bad **m** command can be a

mess. Doing the job one step at a time makes it easier for you to verify at each step that you accomplished what you wanted. It is also a good idea to issue a **w** command before doing anything complicated; then if you make a mistake, you have the file stored in its original state so you can start again.

For more information on moving text, see Section A.4.18.

A.4.15 Performing Global Commands: **g** and **v**

The “global” commands **g** and **v** are used to execute one or more editing commands on all lines that either contain (**g**) or don’t contain (**v**) a specified pattern.

For example, the command

```
g/XENIX/p
```

prints all lines that contain the word “XENIX”. The pattern that goes between the slashes can be anything that could be used in a line search or in a substitute command; exactly the same rules and limitations apply.

For example,

```
g/^./p
```

prints all the *troff* formatting commands in a file (lines that begin with “.”). For an explanation of the use of the caret (^) and the backslash (\), see Section A.5.

The **v** command is identical to **g**, except that it operates on those lines that do *not* contain an occurrence of the pattern. (Mnemonically, the “v” can be thought of as part of the word “inverse”.)

For example

```
v/^./p
```

prints all the lines that don’t begin with a period (i.e., the actual text lines).

Any command can follow **g** or **v**. For example, the following command deletes all lines that begin with “.”:

```
g/^./d
```

This command deletes all empty lines:

```
g/^$/d
```

Probably the most useful command that can follow a global command is the substitute command. For example, we could change the word “Xenix” to “XENIX” everywhere, and verify that it really worked, with

```
g/Xenix/s//XENIX/gp
```

Notice that we used // in the substitute command to mean “the previous pattern,” in this case, “Xenix”. The **p** command executes on each line that matches the pattern, not just on those in which a substitution took place.

The global command makes two passes over the file. On the first pass, all lines that match the pattern are marked. On the second pass, each marked line is examined in turn, dot is set to that line, and the command executed. This means that it is possible for the command that follows a **g** or **v** command to use addresses, set dot, and so on, quite freely. For example:

```
g/^.P/+
```

prints the line that follows each “.P” command (the signal for a new paragraph in some formatting packages). Remember that plus (+) means “one line past dot.” And

```
g/topic/?^.H?p
```

searches for each line that contains the word “topic”, scans backwards until it finds a line that begins with a “.H” (a heading) and prints it, thus showing the headings under which “topic” is mentioned. Finally

```
g/^.EQ+/,^.EN/-p
```

prints all the lines that lie between lines beginning with “.EQ” and “.EN” formatting commands.

The **g** and **v** commands can also be preceded by line numbers, in which case the lines searched are only those in the range specified.

It is possible to give more than one command under the control of a global command. For example, suppose the task is to change “x” to “y” and “a” to “b” on all lines that contain “thing”. Then

```
g/thing/s/x/y\  
s/a/b/
```

is sufficient. The backslash (\) signals the **g** command that the set of commands continues on the next line; the **g** command terminates on the first line that does not end with a backslash.

Note that you cannot use a substitute command to insert a new line within a **g** command.

The command

```
g/x/s/y\  
s/a/b/
```

does *not* work as you might expect. The remembered pattern is the last pattern that was actually executed, so sometimes it will be “x” (as expected), and sometimes it will be “a” (not expected). You must spell it out, like this:

```
g/x/s/x/y\  
s/a/b/
```

It is also possible to execute **a**, **c**, and **i** commands as part of a global command. As with other multiline constructions, add a backslash at the end of each line except the last. Thus, to add an “.nf” and “.sp” command before each “.EQ” line, type

```
g/^\.EQ/i\  
.nf\  
.sp
```

There is no need for a final line containing a period (.) to terminate the **i** command, unless there are further commands to be executed under the global command.

A.4.16 Displaying Tabs and Control Characters: **l**

Ed provides two commands for printing the contents of the text you are editing. You should already be familiar with **p**, in combinations like

```
1,$p
```

to print all the lines you are editing, or

```
s/abc/def/p
```

to change “abc” to “def” on the current line. Less familiar is the “list” (**l**) command which gives slightly more information than **p**. In particular, **l** makes visible characters that are normally invisible, such as tabs and backspaces. If you list a line that contains some of these, **l** prints each tab as “>” and each backspace as “<”. This makes it much easier to correct the sort of typing mistake that inserts extra spaces adjacent to tabs, or inserts a backspace followed by a space.

The **l** command also “folds” long lines for printing. Any line that exceeds 72 characters is printed on multiple lines; each printed line except the last is terminated by a backslash (\), so you can tell it was folded. This is useful for printing lines longer than the width of your terminal screen.

Occasionally, the **l** command will print a string of numbers preceded by a backslash, such as \07 or \16. These combinations are used to make visible characters that normally don't print, like form feed, vertical tab, or bell. Each backslash-number combination represents a single ASCII character. Note that numbers are octal and not decimal. When you see such characters, be wary: they may have surprising meanings when printed on some terminals. Often their presence indicates an error in typing, because they are rarely used.

A.4.17 Undoing Commands: **u**

Occasionally you will make a substitution in a line, only to realize too late that it was a mistake. The **undo(u)** command, lets you “undo” the last substitution. Thus the last line that was substituted can be restored to its previous state by typing

```
u
```

This command does *not* work with the **g** and **v** commands.

A.4.18 Marking Your Spot in a File: **k**

The mark command, **k**, provides a facility for marking a line with a particular name, so that you can later reference it by name, regardless of its actual line number. This can be handy for moving lines and keeping track of them as they move. For example

```
kx
```

marks the current line with the name “x”. If a line number precedes the **k**, that line is marked. (The mark name must be a single lowercase letter.) You can refer to the marked line with the notation

```
^x
```

Note the use of the single, close quotation mark (`^`) here. Marks are very useful for moving things around. Find the first line of the block to be moved and then mark it with

```
ka
```

Then find the last line and mark it with

kb

Go to the place where the text is to be inserted and type:

'a,'bm.

A line can have only one mark name associated with it at any given time.

A.4.19 Transferring Lines: t

We mentioned earlier the idea of saving lines that are hard to type or used often, to cut down on typing time. **Ed** provides another command, called **t** (for transfer) for making a copy of a group of one or more lines at any point. This is often easier than writing and reading.

The **t** command is identical to the **m** command, except that instead of moving lines it simply duplicates them at the place you named. Thus

1,\$t\$

duplicates the entire contents that you are editing.

A common use for **t** is to create a series of lines that differ only slightly. For example, you can type

```
a
Now is the time for all good men to come to the aid of their party.
.
t.                                [make a copy]
s/men/women/                     [change it a bit]
t.                                [make third copy]
s/Now is/yesterday was/          [change it a bit]
```

Your file will look like this:

```
Now is the time for all good men to come to the aid of their party.
Now is the time for all good women to come to the aid of their party.
Yesterday was the time for all good women to come to the aid of their party.
```

A.4.20 Escaping to the Shell: !

Sometimes it is convenient to temporarily escape from the editor to execute a XENIX command. The shell escape (!) command, provides a way to do this.

If you type

!command

your current editing state is suspended, and the XENIX command you asked for is executed. When the command finishes, **ed** will signal you by printing another exclamation (!); at that point you can resume editing.

A.5 Context and Regular Expressions

You may have noticed that things don't work right when you use characters such as the period (.), the asterisk (*), and the dollar sign (\$) in context searches and with the substitute command. **Ed** treats these characters as special. For instance, in a context search or the first string of the substitute command, the period (.) means "any character", not a period, so

/x.y/

means a line with an "x", any character, and a "y", not just a line with an "x", a period, and a "y". The following characters are considered special:

^	caret
.	period
\$	dollar sign
[]	brackets
*	asterisk
\	backslash
/	slash

The next few subsections discuss how to use these characters to describe *regular expressions* — patterns of text in search and substitute commands.

Recall that a trailing **g** after a substitute command causes all occurrences to be changed. With

s/this/that/

and

s/this/that/g

the first command replaces the *first* "this" on the line with "that." If there is more than one "this" on the line, the second form with the trailing **g** changes

all of them.

Either form of the **s** command can be followed by **p** or **l** to print or list the contents of the line. For example, all of the following command lines are legal and mean slightly different things:

```
s/this/that/p
s/this/that/l
s/this/that/gp
s/this/that/gl
```

Make sure you know what the differences are.

Of course, any **s** command can be preceded by one or two line numbers to specify that the substitution is to take place on a group of lines. Thus

```
1,$s/mispell/misspell/
```

changes the *first* occurrence of “mispell” to “misspell” in each line of the file. But

```
1,$s/mispell/misspell/g
```

changes *every* occurrence in each line (and this is more likely to be what you wanted).

If you add a **p** or **l** to the end of any of these substitute commands, only the last line changed is printed, not all the lines. We will talk later about how to print all the lines that were modified.

A.5.1 Period: (.)

On the left side of a substitute command, or in a search, a period stands for *any* single character. Thus the search

```
/x.y/
```

finds any line where “x” and “y” occur separated by a single character, as in

```
x+y
x-y
x y
xzy
```

Since a period matches a single character, it gives you a way to deal with funny characters printed by **l**. Suppose you have a line that appears as

```
th\07is
```

when printed with the **l** command, and that you want to get rid of the `\07`, which represents an ASCII bell character.

The most obvious solution is to try

```
s\07//
```

but this will fail. Another solution is to retype the entire line. This is guaranteed, and is actually quite a reasonable tactic if the line in question isn't too long. But for a very long line, retyping is not the best solution. This is where the metacharacter `“.”` comes in handy. Since `\07` really represents a single character, if we type

```
s/th.is/this/
```

the job is done. The period matches the mysterious character between the `“h”` and the `“i”`, whatever it is.

Since the period matches any single character, the command

```
s/. ./,
```

converts the first character on a line into a comma (`,`), which very often is not what you intended. The special meaning of the period can be removed by preceding it with a backslash.

As is true of many characters in `ed`, the period (`.`) has several meanings, depending on its context. This line shows all three:

```
.s/. ./
```

The first period refers to the line number of the line we are editing, which is called `“dot”`. The second period refers to a metacharacter that matches any single character on that line. The third period is the only one that really is an honest, literal period. (Remember that a period is also used to terminate input from the `i` commands.) On the *right* side of a substitution, the period (`.`) is not special. If you apply this command to the line

```
Now is the time.
```

the result is

```
.ow is the time.
```

which is probably not what you intended. To change the period at the end of the sentence to a comma, type

`s\././`

The special meaning of the period can be removed by preceding it with a backslash. Use of the backslash is discussed in Section A.5.2.

A.5.2 Backslash: \

Since a period means “any character,” the question naturally arises: what do you do when you really want a period? For example, how do you convert the line

Now is the time.

into

Now is the time?

The backslash (\) turns off any special meaning that the next character might have; in particular, “\.” converts the “.” from a “match anything” into a literal period, so you can use it to replace the period in “Now is the time.” like this:

`s\./?/`

The pair of characters “\.” is considered by ed to be a single real period.

The backslash can also be used when searching for lines that contain a special character. Suppose you are looking for a line that contains

.DE

The search

`/.DE/`

isn't adequate, for it will find lines like

JADE
FADE
MADE

because the “.” matches the letter “A” on each of the lines in question. But if you type

`^.DE/`

only lines that contain “.DE” are found.

The backslash can be used to turn off special meanings for characters other than the period. For example, consider finding a line that contains a backslash. The search

```
\/
```

won't work, because the backslash (\) isn't a literal backslash, but instead means that the second slash (/) no longer delimits the search. By preceding a backslash with another backslash, you can search for a literal backslash:

```
\\
```

You can search for a forward slash (/) with

```
//
```

The backslash turns off the special meaning of the slash immediately following so that it doesn't terminate the slash-slash construction prematurely.

A miscellaneous note about backslashes and special characters: you can use any character to delimit the pieces of an s command; there is nothing sacred about slashes. (But you must use slashes for context searching.) For instance, in a line that contains several slashes already, such as

```
//exec //sys.fort.go // etc...
```

you could use a colon as the delimiter. To delete all the slashes, type

```
s/::g
```

The result is:

```
exec sys.fort.go etc...
```

When you are adding text with **a**, or **i** or **c**, the backslash has no special meaning, and you should put in only one backslash for each one you want.

A.5.3 Dollar Sign: \$

The dollar sign "\$", stands for "the end of the line." Suppose you have the line

Now is the

and you want to add the word "time" to the end. Use the dollar sign (\$) like this:

s/\$/ time/

to get

Now is the time

A space is needed before “time” in the substitute command, or you will get

Now is thetime

You can replace the second comma in the following line with a period without altering the first.

Now is the time, for all good men,

The command needed is

s/,\$/./

to get

Now is the time, for all good men.

The dollar sign (\$) here provides context to make specific which comma we mean. Without it the s command would operate on the first comma to produce:

Now is the time. for all good men,

To convert

Now is the time.

into

Now is the time?

as we did earlier, we can use

s/.\$/?/

Like the period (.), the dollar sign (\$) has multiple meanings depending on context. In the following line

`$s/$$/`

the first “\$” refers to the last line of the file, the second refers to the end of that line, and the third is a literal dollar sign to be added to that line.

A.5.4 Caret: ^

The caret (^) stands for the beginning of the line. For example, suppose you are looking for a line that begins with “the”. If you simply type

`/the/`

you will probably find several lines that contain “the” in the middle before arriving at the one you want. But with

`/^the/`

you narrow the context, and thus arrive at the desired line more easily.

The other use of the caret (^) enables you to insert something at the beginning of a line. For example

`s/^/ /`

places a space at the beginning of the current line.

Metacharacters can be combined. To search for a line that contains *only* the characters

`.P`

you can use the command

`/^\.P$/`

A.5.5 Asterisk: *

Suppose you have a line that looks like this:

`text x y text`

where *text* stands for lots of text, and there are an indeterminate number of spaces between the “x” and the “y”. Suppose the job is to replace all the spaces between “x” and “y” with a single space. The line is too long to re-type, and there are too many spaces to count.

This is where the metacharacter “asterisk” (*) comes in handy. A character

followed by a asterisk stands for as many consecutive occurrences of that character as possible. To refer to all the spaces at once, type

```
s/x *y/x y/
```

The “*” means “as many spaces as possible.” Thus “x *y” means an “x”, as many spaces as possible, then a “y”.

The asterisk can be used with any character, not just a space. If the original example were

```
text x-----y text
```

then all minus signs (–) can be replaced by a single space with the command

```
s/x–*y/x y/
```

Finally, suppose that the line were

```
text x.....y text
```

If you blindly type

```
s/x.*y/x y/
```

the result is unpredictable. If there are no other x’s or y’s on the line, the substitution will work, but not necessarily. The period matches *any* single character so the “.*” matches as many single characters as possible, and unless you are careful, it can remove more of the line than you expected. For example, if the line were

```
x text x.....y text y
```

then typing

```
s/x.*y/x y/
```

takes everything from the *first* “x” to the *last* “y”, which, in this example, is undoubtedly more than you wanted.

The solution is to turn off the special meaning of the period (.) with the backslash (\):

```
s/x\.*y/x y/
```

Now the substitution works, for “\.*” means “as many periods as possible”.

There are times when the pattern “.*” is exactly what you want. For example, to change

```
Now is the time for all good men ....
```

into

```
Now is the time.
```

use “.*” to remove everything after the “for”:

```
s/for.*./
```

There are a couple of additional pitfalls associated with the asterisk (*). Most notable is the fact that “as many as possible” means *zero* or more. The fact that zero is a legitimate possibility is sometimes rather surprising. For example, if our line contained

```
xy text x y text
```

and we said

```
s/x *y/x y/
```

the first “xy” matches this pattern, for it consists of an “x”, zero spaces, and a “y”. The result is that the substitute acts on the first “xy”, and does not touch the later one that actually contains some intervening spaces.

The way around this is to specify a pattern like

```
/x *y/
```

which says an “x”, a space, then as many more spaces as possible, then a “y”, in other words, one or more spaces.

The other pitfall associated with the asterisk (*) again relates to the fact that zero is a legitimate number of occurrences of something followed by an asterisk. The command

```
s/x*/y/g
```

when applied to the line

```
abcdef
```

produces

yaybycydyeyfy

which is almost certainly not what was intended. The reason for this is that zero is a legitimate number of matches, and there are no x's at the beginning of the line (so that gets converted into a "y"), nor between the "a" and the "b" (so that gets converted into a "y"), and so on. If you don't want zero matches, use

```
s/xx*/y/g
```

since "xx*" is one or more x's.

A.5.6 Brackets: [and]

Suppose that you want to delete any numbers that appear at the beginning of all lines of a file. You might try a series of commands like

```
1,$s/^1*//
1,$s/^2*//
1,$s/^3*//
```

and so on, but this is clearly going to take forever if the numbers are at all long. Unless you want to repeat the commands over and over until finally all the numbers are gone, you must get all the digits on one pass. That is the purpose of the brackets.

The construction

```
[0123456789]
```

matches any single digit—the whole thing is called a *character class*. With a character class, the job is easy. The pattern "[0123456789]*" matches zero or more digits (an entire number), so

```
1,$s/^ [0123456789]*//
```

deletes all digits from the beginning of all lines.

Any characters can appear within a character class, and there are only three special characters (^,], and -) inside the brackets; even the backslash doesn't have a special meaning. To search for special characters, for example, you can type

```
/[.\$^[]/
```

It's a nuisance to have to spell out the digits, so you can abbreviate them as [0-9]; similarly, [a-z] stands for the lowercase letters, and [A-Z] for uppercase.

Within [], the “[” is not special. To get a “[” (or a “[–” into a character class, make it the first character.

You can also specify a class that means “none of the following characters.” This is done by beginning the class with a caret (^). For example

```
[^0-9]
```

stands for “any character *except* a digit”. Thus, you might find the first line that doesn't begin with a digit with a search like

```
/[^0-9]/
```

Within a character class, the caret has a special meaning only if it occurs at the beginning. Just to convince yourself, verify that

```
/^[^]/
```

finds a line that doesn't begin with a caret.

A.5.7 Ampersand: &

To save typing, use the ampersand (&) to signify the string of text that was found on the left side of a substitute command. Suppose you have the line

```
Now is the time
```

and you want to make it

```
Now is the best time
```

You can type

```
s/the/the best/
```

It's unnecessary to repeat the word “the”. The ampersand (&) eliminates this repetition. On the *right* side of a substitution, the ampersand means “whatever was just matched”, so you can type

```
s/the/& best/
```

and the ampersand will stand for “the”. This isn't much of a saving if the thing matched is just “the”, but if the match is very long, or if it is something like “.*” which matches a lot of text, you can save some tedious typing. There is also much less chance of making a typing error in the replacement text. For example, to put parentheses in a line, regardless of its length, type

```
s././(&)/
```

The ampersand can occur more than once on the right side. For example

```
s/the/& best and & worst/
```

makes

```
Now is the best and the worst time
```

and

```
s././&? &!!/
```

converts the original line into

```
Now is the time? Now is the time!!
```

To get a literal ampersand use the backslash to turn off the special meaning. For example

```
s/ampersand\&/
```

converts the word into the symbol. The ampersand is not special on the left side of a substitute command, only on the right side.

A.5.8 Substituting New Lines

Ed provides a facility for splitting a single line into two or more shorter lines by substituting in a newline. For example, suppose a line has become unmanageably long because of editing. If it looks like

```
text xy text
```

you can break it between the “x” and the “y” like this:

```
s/xy/x\  
y/
```

This is actually a single command, although it is typed on two lines. Because the backslash (\) turns off special meanings, a backslash at the end of a line makes the newline there no longer special.

You can in fact make a single line into several lines with this same mechanism. As an example, consider italicizing the word “very” in a long line by splitting “very” onto a separate line, and preceding it with the formatting

ccommand “.I”. Assume the line in question looks like this:

```
text a very big text
```

The command

```
s/ very /\n.\nvery\  
/
```

converts the line into four shorter lines, preceding the word “very” with the line “.I”, and eliminating the spaces around the “very” at the same time.

When a new line is substituted in a string, dot is left at the last line created.

A.5.9 Joining Lines

Lines may be joined together, with the **j** command. Assume that you are given the lines

```
Now is  
the time
```

Suppose that dot is set to the first line. Then the command

```
j
```

joins them together to produce

```
Now is the time
```

No spaces are added, which is why a space was shown at the beginning of the second line.

All by itself, a **j** command joins the lines signified by dot and dot + 1, but any contiguous set of lines can be joined. Just specify the starting and ending line numbers. For example

```
1,$_p
```

joins all the lines in a file into one big line and prints it.

A.5.10 Rearranging a Line: \ (and \)

Recall that “&” is shorthand for whatever was matched by the left side of an s command. In much the same way, you can capture separate pieces of what was matched. The only difference is that you have to specify on the left side

just what pieces you're interested in.

Suppose that you have a file of lines that consist of names in the form

Smith, A. B.
Jones, C.

and so on, and you want the initials to precede the name, as in:

A. B. Smith
C. Jones

It is possible to do this with a series of editing commands, but it is tedious and error-prone.

The alternative is to “tag” the pieces of the pattern (in this case, the last name, and the initials), then rearrange the pieces. On the left side of a substitution, if part of the pattern is enclosed between `\(` and `\)`, whatever matched that part is remembered, and available for use on the right side. On the right side, the symbol “`\1`” refers to whatever matched the first `\(...\)` pair; “`\2`”, to the second `\(...\)`, and so on.

The command

```
1,$s/^\([.*]\),*(.*)\2\1/
```

although hard to read, does the job. The first `\(...\)` matches the last name, which is any string up to the comma; this is referred to on the right side with “`\1`”. The second `\(...\)` is whatever follows the comma and any spaces, and is referred to as “`\2`”.

With any editing sequence this complicated, it's unwise to simply run it and hope. The global commands **g** and **v** provide a way for you to print exactly those lines which were affected by the substitute command, and thus verify that it did what you wanted in all cases.

A.6 Speeding Up Editing

One of the most effective ways to speed up your editing is knowing what lines will be affected by a command if you don't specify the lines it is to act on, and on what line you will be positioned (i.e., the value of dot) when a command finishes. If you can edit without specifying unnecessary line numbers, you can save a lot of typing.

For example, if you issue a search command like

```
/thing/
```

you are left pointing at the next line that contains “thing”. Then no address is required with commands like **s** to make a substitution on that line, or **p** to print it, or **l** to list it, or **d** to delete it, or **a** to append text after it, or **c** to change it, or **i** to insert text before it.

What happens if there is no occurrence of “thing”? Dot is unchanged. This is also true if the cursor was on the only occurrence of “thing” when you issued the command. The same rules hold for searches that use *?...?*; the only difference is the direction in which you search.

The delete command, **d**, leaves dot pointing at the line that followed the last deleted line. When the line dollar (\$) gets deleted, however, dot points at the *new* line \$.

The line-changing commands **a**, **c**, and **i**, by default, all affect the current line. If you give no line number with them, **a** appends text after the current line, **c** changes the current line, and **i** inserts text before the current line.

The **a**, **c**, and **i** commands behave identically in one respect — when you stop appending, changing or inserting, dot points at the last line entered. This is exactly what you want when typing and editing on the fly. For example, you can type

```
a
text
botch (minor error)
.
s/botch/correct/ (fix botched line)
a
more text
.
```

without specifying any line number for the substitute command or for the second append command. Or you can type

```
a
text
horrible botch (major error)
.
c          (replace entire line)
fixed up line
.
```

Experiment to determine what happens if you add *no* lines with an **a**, **c**, or **i** command.

The **r** command reads a file into the text being edited, at the end if you give no

address, or after the specified line if you do. In either case, dot points at the last line read in. Remember that you can even type

Or

to read a file in at the beginning of the text. (You can also type *0a* or *li* to start adding text at the beginning.)

The **w** command writes out the entire file. If you precede the command by one line number, that line is written out. If you precede it by two line numbers, that range of lines is written out. The **w** command does *not* change dot: the current line remains the same, regardless of what lines are written out. This is true even if you type something like

```
/^\.AB/,/^\.AE/w abstract
```

which involves a context search.

(Since the **w** command is so easy to use, you should save what you are editing regularly as you go along just in case the system crashes, or in case you accidentally delete what you're editing.)

The general rule is simple: you are left sitting on the last line changed; if there were no changes, then dot is unchanged. To illustrate, suppose there are three lines in the buffer, and the line given by dot is the middle one:

```
x1
x2
x3
```

Then the command

```
-,+s/x/y/p
```

prints the third line, which is the last one changed. But if the three lines had been

```
x1
y2
y3
```

and the same command had been issued while dot pointed at the second line, only the first line would be changed and printed, and that is where dot would be set.

A.6.1 Semicolon: ;

Searches with `/.../` and `?...?` start at the current line and move forward or backward, respectively, until they either find the pattern or return to the current line. Sometimes this is not what you want. Suppose, for example, that the buffer contains lines like this:

```
.
.
.
ab
.
.
bc
.
.
.
```

Starting at line 1, you would expect the command

```
/a/,b/p
```

to print all the lines from the “ab” to the “bc” inclusive. This is not what happens. *Both* searches (for “a” and for “b”) start from the same point, and thus they both find the line that contains “ab”. As a result, a single line is printed. Worse, if there had been a line with a “b” in it before the “ab” line, then the print command would be in error, since the second line number would be less than the first, and it is illegal to try to print lines in reverse order.

This is because the comma separator for line numbers doesn't set dot as each address is processed; each search starts from the same place. In `ed`, the semicolon (;) can be used just like the comma, with the single difference that use of a semicolon forces dot to be set at the time the semicolon is encountered, as the line numbers are being evaluated. In effect, the semicolon “moves” dot. Thus, in our example above, the command

```
/a/;/b/p
```

prints the range of lines from “ab” to “bc”, because after the “a” is found, dot is set to that line, and then “b” is searched for, starting beyond that line.

This property is most often useful in a very simple situation. Suppose you want to find the *second* occurrence of “thing”. You could type

```
/thing/
//
```

but this prints the first occurrence as well as the second, and is a nuisance when you know very well that it is only the second one you're interested in. The solution is to type

```
/thing;/
```

This says “find the first occurrence of ‘thing’, set dot to that line, then find the second occurrence and print only that.”

Closely related is searching for the second to last occurrence of something, as in

```
?something?;??
```

Finally, bear in mind that if you want to find the first occurrence of something in a file, starting at an arbitrary place within the file, it is not sufficient to type

```
1;/thing/
```

because if “thing” occurs on line 1 it won't be found. The command

```
0;/thing/
```

will work because it starts the search at line 1. This is one of the few places where 0 is a legal line number.

A.6.2 Interrupting the Editor

As a final note on what dot gets set to, you should be aware that if you press the INTERRUPT key while **ed** is executing a command, your file is restored, as much as possible, to what it was before the command began. Naturally, some changes are irrevocable — if you are reading in or writing out a file, making substitutions, or deleting lines. These will be stopped in some unpredictable state in the middle (which is why it is usually unwise to stop them). Dot may or may not be changed.

If you are using the **print** command, dot is not changed until the printing is done. Thus, if you decide to print until you see an interesting line, and then press INTERRUPT, to stop the command, dot will *not* be set to that line or even near it. Dot is left where it was when the **p** command was started.

A.7 Cutting and Pasting with the Editor

This section describes how to manipulate pieces of files, individual lines or groups of lines.

A.7.1 Inserting One File into Another

Suppose you have a file called *memo*, and you want the file called *table* to be inserted just after a reference to Table 1. That is, in *memo* somewhere is a line that says

Table 1 shows that ...

and the data contained in *table* has to go there.

To put *table* into the correct place in the file edit *memo*, find “Table 1”, and add the file *table* right there:

```
ed memo
/ Table 1/
response from ed
.r table
```

The critical line is the last one. The **r** command reads a file; here you asked for it to be read in right after line dot. An **r** command, without any address, adds lines at the end, so it is the same as “\$r”.

A.7.2 Writing Out Part of a File

The other side of the coin is writing out part of the document you're editing. For example, you may want to split the table from the previous example out into a separate file so it can be formatted and tested separately. Suppose that in the file being edited we have

```
.TS
[lots of stuff]
.TE
```

which is the way a table is set up for the **tbl** program. To isolate the table in a separate file called *table*, first find the start of the table (the “.TS” line), then write out the interesting part. For example, first type

```
/^\.TS/
```

This prints out the found line:

```
.TS
```

Next type

```
./^\.TE/w table
```

and the job is done. If you are confident, you can do it all at once with

/^\.TS/;/^\.TE/w table

The point is that the `w` command can write out a group of lines, instead of the whole file. In fact, you can write out a single line if you like; just give one line number instead of two. If you have just typed a horribly complicated line and you know that it (or something like it) is going to be needed later, then save it—don't retype it. For example, in the editor, type

```
a
lots of stuff
horrible line
.
.w temp
a
more stuff
.
.r temp
a
more stuff
.
```

A.8 Editing Scripts

If a fairly complicated set of editing operations is to be performed on a whole set of files, the easiest thing to do is to make up a *script*, i.e., a file that contains the operations you want to perform, then apply this script to each file in turn.

For example, suppose you want to change every “Xenix” to “XENIX” and every “USA” to “America” in a large number of files. Put the following lines into the file *script*:

```
g/Xenix/s//XENIX/g
g/USA/s//America/g
w
q
.
```

Now you can type

```
ed -file1 <script
ed -file2 <script
...
```

This causes `ed` to take its commands from the prepared file *script*. Notice that the whole job has to be planned in advance, and that by using the XENIX shell command interpreter, you can cycle through a set of files automatically. The dash (`-`) suppresses unwanted messages from `ed`.

When preparing editing scripts, you may need to place a period as the only character on a line to indicate termination of input from an **a** or **i** command. This is difficult to do in **ed**, because the period you type will terminate input rather than be inserted in the file. Using a backslash to escape the period won't work either. One solution is to create the script using a character such as the at-sign (@) to indicate end of input. Then, later, use the following command to replace the at-sign with a period:

```
s/^@$/./
```

A.9 Summary of Commands

The following is a list of all **ed** commands. The general form of **ed** commands is the command name, preceded by one or two optional line numbers and, in the case of **e**, **f**, **r**, and **w**, followed by a filename. Only one command is allowed per line, but a **p** command may follow any other command (except **e**, **f**, **r**, **w**, and **q**).

- a** Appends, i.e., adds lines to the buffer (at line dot, unless a different line is specified). Appending continues until a period is typed on a new line. The value of dot is set to the last line appended.
- c** Changes the specified lines to the new text which follows. The new lines are terminated by a period on a new line, as with **a**. If no lines are specified, replace line dot. Dot is set to the last line changed.
- d** Deletes the lines specified. If none are specified, deletes line dot. Dot is set to the first undeleted line following the deleted lines unless dollar (\$) is deleted, in which case dot is set to dollar.
- e** Edits a new file. Any previous contents of the buffer are thrown away, so issue a **w** command first.
- f** Prints the remembered filename. If a name follows **f**, then the remembered name is set to it.
- g** The command **g/string/commands** executes *commands* on those lines that contain *string*, which can be any context search expression.
- i** Inserts lines before specified line (or dot) until a single period is typed on a new line. Dot is set to the last line inserted.

- l** Lists lines, making visible nonprinting ASCII characters and tabs. Otherwise similar to **p**.
- m** Moves lines specified to after the line named after **m**. Dot is set to the last line moved.
- p** Prints specified lines. If none are specified, print the line specified by dot. A single line number is equivalent to a **line-number(p)** command. A single RETURN prints “.+1”, the next line.
- q** Quits **ed**. Your work is not saved unless you first use the **w** command. Press “q” twice to terminate an edit.
- r** Reads a file into the buffer (at the end unless specified elsewhere.) Dot is set to the last line read.
- s** The command “s/string1/string2/” substitutes the pattern matched by *string1* with the string specified by *string2* in the specified lines. If no lines are specified, the substitution takes place only on the line specified by dot. Dot is set to the last line in which a substitution took place, which means that if no substitution takes place, dot remains unchanged. The **s** command changes only the first occurrence of *string1* on a line; to change multiple occurrences on a line, type a **g** after the final slash.
- t** Transfers specified lines to the line named after **t**. Dot is set to the last line moved.
- v** The command **v/string/commands** executes *commands* on those lines that do not contain *string*.
- u** Undoes the last substitute command.
- w** Writes out the editing buffer to a file. Dot remains unchanged.
- .=** Prints value of dot. (An equal sign by itself prints the value of \$.)
- !cmd-line** The line **!cmd-line** causes *cmd-line* to be executed as a XENIX command.
- /string/** Context search. Searches for next line which contains this string of characters and prints it. Dot is set to the line where string was found. The search starts at .+1, wraps around from \$ to 1, and continues to dot, if necessary.

?string? Context search in reverse direction. Starts search at *.-1* , scans to 1, wraps around to *\$*.

Index - User's Guide

- . command
 - vi use See Vi
 - vi 5-3
- { } command See Braces command { }
- : command See Colon command (:)
- . command See Dot command (.)
- ! command See escape command (!)
- / command See Vi
- * See Asterisk (*)
- [] See Brackets ([])
- See Dash (-)
- > See Greater-than sign (>)
- . See Period (.)
- ? See Question mark (?)
- / See Slash (/)
- \$# variable, argument recording 7-13
- \$? variable, command exit status 7-14
- \$- variable, execution flags 7-14
- \$! variable, process number 7-14
- \$\$ variable, process number 7-14
- 0 command See Vi
- a character, permission change 4-17
- a command
 - mail 6-11, 6-18, 6-30
- a operator 7-35
- Absolute pathname See Pathname
- Account, new user 2-1
- Addition See BC
- Alias command See a command
- Alphabetizing See sort command
- Ampersand (&)
 - and-if operator symbol See And-if operator (& &)
 - background command 3-7
 - background process 4-22, 7-20, 7-54
 - command list 7-19
 - ed use See Ed
 - interrupt, quit immunity 7-20
 - jobs to other computers 7-20
 - Ampersand (&) (*continued*)
 - metacharacter See Ed
 - off-line printing 7-20
 - use restraint 7-20
- And-if operator (& &)
 - command list 7-19
 - description, use 7-20
 - designated 7-54
- Append
 - output append symbol See Output
- Appending files 4-7
- Appending See Output
- Argument
 - filename 7-3
 - list creation 7-3
 - mail commands 6-8
 - number checking, \$# variable 7-13
 - processing 7-17
 - redirection argument location 7-8
 - shell argument passing 7-17
 - substitution sequence 7-18
 - switch See Switch
 - test command argument 7-35
- Arithmetic
 - See also BC
 - expr command effect 7-36
- askcc option See Mail
- asksubject option See Mail
- Asterisk (*)
 - BC
 - comment convention 8-13, 8-14
 - multiplication operator symbol 8-2, 8-4
 - directory name, use avoidance 7-4
 - filename, use avoidance 3-4
 - filename wildcard 3-6
 - mail
 - character matching 6-7
 - message saved, header notation 6-14, 6-16
 - metacharacter 7-54

Asterisk (*) (*continued*)

See also Asterisk (*)

special shell variable 7-18
asterisk (*)

ed metacharacter See Ed

at command 4-21

at -r command 4-22

At sign (@), mail 6-27, 6-37

auto command, BC 8-19

autombox option See Mail

autoprint option See Mail

b command See Vi

-b option

mail 6-27

Background process 4-22, 4-23

\$! variable 7-14

ampersand (&) operator 4-22, 7-20,
7-54

CTRL-D immunity 7-20

dial-up line

CTRL-D effect 7-20

nohu command 7-20

INTERRUPT immunity 7-20

logout immunity 7-20

QUIT immunity 7-20

use restraint 7-20

Backslash ()

BC

comment convention 8-13, 8-14

line continuation notation 8-6

ed See Ed

line continuation notation 7-46

metacharacter escape 7-4

quoting 7-54

BACKSPACE key

erasure function 2-4

mail 6-6, 6-10

Batch processing See Command

bc command

BC

file reading, execution 8-13

invocation `bc -i`

calculation 4-31

bc -l command, BC 8-13

BC

addition operator

evaluation order 8-15

left to right binding 8-4

scale 8-6, 8-17

symbol (+) 8-4

additive operators

See also Specific Operator

left to right binding 8-17

alphabetic register See storage
register

arctan function

availability 8-1

loading procedure 8-13

array

auto array 8-19

characteristics 8-14

identifier 8-14, 8-19

named expression 8-15

name 8-9

one-dimensional 8-9

assignment operator

designated, use 8-18

evaluation order 8-15

positioning effect 8-5

symbol (=) 8-5

assignment statement 8-12

asterisk (*)

comment convention 8-13, 8-14

multiplication operator symbol 8-2,
8-4

auto command 8-19

auto, keyword 8-14

auto statement

built-in statement 8-19

backslash ()

comment convention 8-13, 8-14

line continuation notation 8-6

bases 8-5

bc command

file reading, execution 8-13

invocation 8-1

bc -l command 8-13

Bessel function

Bessel function (*continued*)

- availability 8-1
- loading procedure 8-13
- BKSP key 8-2
- braces ({})
 - compound statement enclosure 8-19
 - function body enclosure 8-8
- brackets ([])
 - array identifier 8-14
 - auto array 8-19
 - subscripted variable 8-9
- break, keyword 8-14
- break statement
 - built-in statement 8-19
- built-in statement 8-19
- caret (^), exponentiation operator
 - symbol 8-4
- command See bc command
- comment convention 8-13, 8-14
- compound statement 8-19
- constant
 - composition 8-15
 - defined 8-15
- construction
 - diagram 8-13
 - space significance 8-13
- control statements 8-10
- cos function
 - availability 8-1
 - loading procedure 8-13
- define, keyword 8-14
- define statement
 - built-in statement 8-19
 - description, use 8-20
- demonstration run 8-1
- description 8-1
- division operator
 - left to right binding 8-4, 8-17
 - scale 8-7, 8-17
 - symbol (/) 8-4
- equal sign (=)
 - assignment operator symbol 8-5
 - relational operator 8-10, 8-18
- equivalent constructions diagram 8-13

BC (*continued*)

- evaluation sequence 8-2
- exclamation point (!)
 - relational operator 8-10, 8-18
- exit 8-2, 8-3
- exponential function
 - availability 8-1
 - loading procedure 8-13
- exponentiation operator
 - right to left binding 8-4, 8-17
 - scale 8-7, 8-17
 - symbol (^) 8-4
- expression
 - enclosure 8-15
 - evaluation order 8-15
 - named expression 8-15
 - statement 8-19
- for, keyword 8-14
- for statement
 - break statement effect 8-19
 - built-in statement 8-19
 - description, use 8-10
 - format 8-20
 - range execution 8-11
 - relational operator 8-18
- function call
 - defined 8-16
 - description 8-16
 - evaluation order 8-15
 - procedure 8-8
 - syntax 8-16
- function
 - argument absence 8-9
 - array 8-9
 - calling See function call
 - definition procedure 8-8
 - form 8-8
 - identifier 8-14
 - name 8-8
 - parameters 8-8
 - return statement See return statement
 - termination, return statement 8-20
 - variable automatic 8-8
- global storage class 8-18

BC (*continued*)

- greater-than sign (>), relational operator 8-10, 8-18
- hexadecimal digit
 - ibase 8-6
 - obase 8-6
 - value 8-15
- ibase
 - decimal input setting 8-6
 - defined 8-15
 - initial setting 8-5
 - keyword 8-14
 - named expression 8-15
 - setting 8-5
 - variable 8-7
- identifier
 - array See array
 - auto statement effect 8-19
 - description 8-14
 - global 8-19
 - local 8-19
 - named expression 8-15
 - value 8-19
- if, keyword 8-14
- if statement
 - built-in statement 8-19
 - description, use 8-10
 - format 8-20
 - range execution 8-10
 - relational operator 8-18
- INTERRUPT key 8-2
- introduction 8-1
- invocation 8-1
- keywords designated 8-14
- language features 8-12
- length
 - built-in function 8-16
 - keyword 8-14
- less-than sign (<), relational operator 8-10, 8-18
- line continuation notation 8-6
- local storage class 8-18
- log function
 - availability 8-1
 - loading procedure 8-13

BC (*continued*)

- math function library See bc -l command
- minus sign (-)
 - subtraction operator symbol 8-4
- minus sign (-)
 - unary operator symbol 8-4, 8-16
- modulo operator
 - left to right binding 8-4, 8-17
 - scale 8-7, 8-17
 - symbol (%) 8-4
- multiplication operator
 - evaluation order 8-15
 - left to right binding 8-4, 8-17
 - scale 8-7, 8-17
 - symbol (*) 8-2, 8-4
- multiplicative operators
 - See also Specific Operator
 - left to right binding 8-17
- named expression 8-15
- negative number, unary minus sign (-) 8-4
- obase
 - conversion speed 8-6
 - defined 8-15
 - description 8-6
 - hexadecimal notation 8-6
 - initial setting 8-6
 - keyword 8-14
 - named expression 8-15
 - variable 8-7
- operator
 - See also Specific Operator
 - designated, use 8-4
- parentheses (())
 - expression enclosure 8-15
 - function identifier argument enclosure 8-14
- percentage sign (%), modulo operator symbol 8-4
- plus sign (+)
 - addition operator symbol 8-4
 - unary operator symbol 8-16
- program flow alteration 8-10
- quit command 8-2, 8-3

BC (*continued*)

- quit, keyword 8-14
- quit statement
 - BC exit 8-20
 - built-in statement 8-19
- quoted string statement 8-19
- register See storage register
- relational operator
 - designated 8-10, 8-18
 - evaluation order 8-15
- RETURN key 8-2
- return, keyword 8-14
- return statement
 - built-in statement 8-19
 - description 8-20
 - form 8-8
- scale command 8-7
- scale
 - addition operator 8-6, 8-17
 - arctan function 8-13
 - Bessel function 8-13
 - built-in function 8-16
 - command See scale command
 - cos function 8-13
 - decimal digit value 8-7
 - defined 8-15
 - description 8-6
 - division operator 8-7, 8-17
 - exponential function 8-13
 - exponentiation operator 8-7, 8-17
 - initial setting 8-7
 - keyword 8-14
 - length function 8-16
 - length maximum 8-6
 - log function 8-13
 - modulo operator 8-7, 8-17
 - multiplication operator 8-7, 8-17
 - named expression 8-15
 - sin function 8-13
 - square root effect 8-7, 8-16
 - subtraction operator 8-6, 8-17
 - value printing procedure 8-7
 - variable 8-7
- semicolon (;), statement separation 8-3, 8-19

BC (*continued*)

- sin function
 - availability 8-1
 - loading procedure 8-13
- slash (/), division operator symbol 8-4
- space significance 8-13
- square root
 - built-in function 8-16
 - keyword 8-14
 - result as integer 8-5
 - scale procedure 8-7
 - sqrt keyword 8-14
- statement
 - See also Specific Statement
 - entry procedure 8-12
 - execution sequence 8-19
 - separation methods 8-19
 - types designated 8-19
- storage classes 8-18
- storage register 8-5
- subscripted variable
 - array See array
 - description 8-9
 - subscript See subscript
- subscript
 - fractions discarded 8-9
 - truncation 8-14
 - value limits 8-9
- subtraction operator
 - left to right binding 8-4
 - scale 8-6, 8-17
 - symbol (-) 8-4
- syntax 8-1
- token composition 8-14
- truncation use when 8-7
- unary operator
 - designated 8-16
 - evaluation order 8-15
 - left to right binding 8-16
 - symbol (-) 8-4
- variable
 - automatic 8-8, 8-19
 - name 8-8
 - subscripted See subscripted variable

- BC (*continued*)
 - while, keyword 8-14
 - while statement
 - break statement effect 8-19
 - built-in statement 8-19
 - description, use 8-10
 - execution 8-21
 - range execution 8-10
 - relational operator 8-18
- `^bcc` escape See Mail
- Bessel function See BC
- /bin directory
 - contents 3-4, 7-33
 - name derivation 7-33
 - /usr/bin duplicate determination 7-44
- Binary file See File
- Binary logical and operator 7-35
- Binary logical or operator 7-35
- BINUNIQ shell procedure 7-44
- BKSP key
 - BC 8-2
 - command-line buffer editing 3-7
- B $\bar{K}$ SP
 - vi cursor movement 5-16
- Block special device 4-14
- Bourne shell
 - TERM variable 5-47
 - terminal type 5-47
- B-aces ({})
 - BC
 - compound statement enclosure 8-19
 - function body enclosure 8-8
 - command grouping 7-26
 - pipeline, command list enclosure 7-21
 - variable
 - conditional substitution 7-39
 - enclosure 7-11
- B-aces command {} 7-41
- Brackets ([])
 - filename, use avoidance 3-4
- Brackets ([])
 - BC
 - array identifier 8-14
 - auto array 8-19
- BC (*continued*)
 - subscripted variable 8-9
 - directory name, use avoidance 7-4
 - ed metacharacter See Ed
 - metacharacter 7-54
 - pattern-matching functions 3-6
 - test command, use in lieu of 7-34
- break command
 - for command control 7-25
 - loop control 7-25
 - shell built-in command 7-41
 - special shell command 7-30
 - while command control 7-25
- Buffers See Vi
- c command See Ed
- C language
 - BC
 - comment convention similarity 8-13
 - syntax agreement 8-1
 - shell language 7-1
 - c option, shell invocation 7-40
 - c option
 - mail 6-27
- C shell
 - TERM variable 5-47
 - terminal type setting 5-47
- cal command 4-30
- Calculation
 - See also BC
- Calculator functions See BC
- calendar command 4-31
- Calendar reminder service 6-28
- cancel *See also* command
 - see
- Caret (^)
 - BC, exponentiation operator symbol 8-4
 - ed use See Ed
 - mail, first message specification 6-6, 6-13, 6-30
- case command
 - description, use 7-22

User's Guide

case command (*continued*)

- exit status 7-23

- redirection 7-27

- shell built-in command 7-41

Case delimiter symbol (;) 7-54

Case significance 2-3

Case-part 7-52

cat command

- file

- combining 4-6

- contents display 2-3

Cat

- command 4-6

~cc escape See Mail

cd arg command 7-30

cd command 4-13

- directory change 3-4, 7-15

- mail 6-19, 6-30

- parentheses use 7-15

- time consumption minimization 7-42

Changing password 4-2

Changing terminal types 4-3

Character class See Ed

Character counting 4-20

Character special device 4-14

chmod command 4-16, 4-17

- directory permission change 3-2

- file permission change 3-2

chron option See Mail

Co command See Vi

Colon (:)

- command See Colon command (:)

- mail

- command escape 6-23

- PATH variable use 7-12

- variable conditional substitution 7-40

- vi use See Vi

Colon command (:)

- description 7-30

- shell built-in command 7-41

- special shell command 7-30

Command line

- ampersand (&) effect 3-7

- buffer defined 3-7

- defined 3-7

Command line (*continued*)

- entry 4-4

- erasure 4-4

- execution 7-19

- interpretation 3-7

- multiple commands entry 3-7

- options

- See also Specific Option

- designated 7-40

- pipeline, use in 7-21

- rescan 7-19

- RETURN key effect 4-4

- scanning sequence 7-18

- substitution 7-9

Command list

- case command, execution 7-22

- defined 7-19

- for command, execution 7-24

- grammar 7-52

Command

- See also Specific Command

- background submittal 3-7

- batch processing See background submittal

- dash (-) use 3-4

- defined 7-19

- delimiter See Ed

- directory See /bin directory

- directory See Directory

- enclosure in parentheses (()), effect 7-41

- environment 7-16

- execution 3-7, 7-2

- RETURN key required 2-2

- sequence 4-22

- time 7-41

- exit status See Exit status

- grammar 7-52

- grouping

- exit status 7-27

- parentheses (()) use 7-54

- procedure 7-26

- WRITEMAIL shell procedure 7-51

- keyword parameter 7-16

- line See Command line

Command (*continued*)

- list See Command list
- lowercase letters 3-8
- mail commands summary 6-30
- multiple commands entry 3-7, 7-8
- name error 2-3
- output substitution symbol 7-54
- private command name 7-3
- program invocation 3-7
- public command name 7-3
- RETURN key required 2-2
- search
 - PATH variable 7-12
 - process 7-42
- separation symbol (;) 7-54
- shell, built-in commands designated 7-41
- simple command
 - defined 7-2, 7-19
 - grammar 7-52
- slash (/) beginning, effect 7-2
- special shell commands See Shell
- special shell commands See Specific Special Command
- substitution
 - back quotation marks (``) 7-4
 - double quotation marks 7-4
 - procedure 7-9
 - redirection argument 7-6
- syntax 3-8
- typing error correction 2-4
- vi commands See Vi

Commands

- at -r 4-22
- at 4-21
- cal 4-30
- cat 4-6, 4-7
- cd 4-13
- copy 4-12
- cp 4-7
- date 4-30
- diff3 4-18
- diff 4-18
- echo 4-18
- find 4-8

Commands (*continued*)

- head 4-6
- lc 4-10
- ln 4-9
- mkdir 4-12
- more 4-5
- mv 4-7
- passwd 4-2
- ps 4-23
- pwd 4-13
- rmdir 4-12
- rm 4-8
- sort 4-19
- stty 4-4
- tail 4-6
- wc 4-20
- Communication See Mail
- Comparing files 4-18
- Compose escapes See Mail
- compose escapes 6-1
- Concatenate See cat command
- Console 2-3
- continue command
 - for command control 7-25
 - shell built-in command 7-41
 - special shell command 7-30
 - until command control 7-25
 - while command control 7-25
- Control characters
 - filename use restrictions 3-4
- Control command
 - See also Specific Control Command
 - redirection 7-27
- Copy command 4-12
- Copying a directory 4-12
- Copying files 4-7
- Copying See cp command
- COPYPAIRS shell procedure 7-44
- COPYTO shell procedure 7-45
- Counting, wc command 4-20
- cp command 4-7
- CR key See RETURN key
- Creating a directory 4-12
- Creating a file 4-4
- CTRL-D

CTRL-D (*continued*)

- background process immunity 7-20
- BC exit 8-2, 8-3
- end-of-file 4-2
- logging out 2-5
- mail 4-29
 - message sending 6-2, 6-9
 - reply message termination 6-11, 6-17
- shell exit 6-19, 7-25

CTRL-G

- vi See Vi 5-10

CTRL-H, mail 6-6

CTRL-Q, output resumption 4-4

CTRL-S, output stopping 4-4

CTRL-U

- command-line buffer editing 3-7
- kill character 2-4
- line kill 4-4
- mail, line killing 6-6, 6-10

Current directory

- change 3-4
 - procedure 4-13
- description 4-13
- printing 4-10
- shorthand name 3-5
- user residence 3-5

Current line

- See Vi

Cursor movement

- vi See Vi

Cutting and pasting procedure See Ed

D command See Vi

d\$ command See Vi

d command

- ed use See Ed
- mail, message deletion See Mail

d0 command See Vi

Dash (-), permission

- denial notation 4-15
- ordinary file notation 4-14

Dash (-)

- command option use 3-4

Dash (-) (*continued*)

- filename, use avoidance 3-4
- switch use 3-8

Date command 4-30

date command 2-2

dd command See Vi

~ dead escape See Mail

Delete buffer See Vi

Deleting a file 4-8

Deletion See d command

Deletion

- vi procedure See Vi

Delimiter See Ed

Demonstration 2-1

/dev directory

- contents 3-5

/dev/console directory

- contents 3-5

Device special file See Special file

Device

- filename 3-3
- filename required 3-3
- pathname 3-3

/dev/tty directory

- contents 3-5

Diagnostic output See Output

Dial-up line See Background process

Diff command 4-18

diff3 4-18

Directory

- access permission See Permission

/bin See /bin directory

changing 4-13

command See cd command

composition 3-2

copying 4-12

creating 4-12

current directory See Current

directory

description 3-2

/dev See /dev directory

diagram 3-3

file See File

filename

- required 3-3

- filename (*continued*)
 - unique to directory 3-3, 3-4
- /lib See /lib directory
- listing 4-11
 - columns 4-10
- long listing 4-11
- name, metacharacter avoidance 7-4
- parent directory See Parent directory
- pathname required 3-3
- permission notation 4-14
- permission See Permission
- protection 3-2
- recursive listing 4-11
- removing 4-12
- renaming 4-12
- search permission See Permission
- search
 - optimum order 7-42
 - PATH variable 7-42
 - sequence change 7-3
 - size effect 7-43
 - time consumption 7-42
- size consideration 7-43
- /tmp directory 4-23
- /tmp See /tmp directory
- /ty See /try directory
- user control 3-2
- /usr See /usr directory
- working directory See Current directory
- Displaying a file 4-5
- DISTINCT1 shell procedure 7-46
- Division See BC 8-4
- Dollar sign (\$)
 - ed use See Ed
 - mail, final message specification 6-6, 6-13, 6-30
 - positional parameter prefix 7-10, 7-11
 - PS1 variable default value 7-13
 - variable prefix 7-11
 - vi See Vi
- Dot (.)
 - command See Dot command (.)
 - ed use See Ed
 - mail, current message specification 6-6, 6-13
- Dot (.) (*continued*)
 - vi use See Vi
- Dot command (.)
 - description, use 7-27
 - shell built-in command 7-41
 - shell procedure alternate 7-33
 - special shell command 7-30
- Dot option See Mail
- Double quotation marks See Quotation marks, double
- dp command See Mail
- DRAFT shell procedure 7-47
- dw command See Vi
- e command
 - ed use See Ed
 - mail 6-6, 6-30
 - mailR 6-18
- e option, shell procedure 7-33
- echo command 2-3, 4-18
 - description, use 7-36
 - mail 6-30
 - n option effect 7-36
 - syntax 7-36
- Ed
 - a command
 - append A-42
 - backslash () characteristics A-26
 - dot (.) setting A-36, A-42
 - global combination A-19
 - input termination A-3, A-24
 - abortion, q command A-43
 - address arithmetic A-6
 - ampersand (&)
 - literal A-33
 - metacharacter A-32
 - substitution A-32
 - asterisk (*), metacharacter A-22, A-28
 - at sign (@), script A-42
 - backslash ()
 - a command A-26
 - c command A-26
 - g command A-18

backslash () (*continued*)

- i command A-26
- line folding A-20
- literal A-26
- metacharacter escape A-25, A-26, A-33
- metacharacter A-22, A-25
- multiline construction A-19
- number string A-20
- v command A-18

backspace printing A-19

brackets ([])

- character class A-31
- metacharacter A-22, A-31

buffer

- writing to file See w command

c command

- backslash () characteristics A-26
- dot (.) setting A-15, A-36, A-42
- global combination A-19
- input termination A-15
- line change A-15, A-42

caret (^)

- character class A-31
- line beginning notation A-28
- metacharacter A-22, A-28

change command See c command

character class A-31

character

- deletion at line beginning A-31

command

- combinations A-18
- delimiter character A-26
- editing command See e command
- form A-42
- INTERRUPT key effect A-39
- listing A-42
- multicommand line restrictions A-11
- summary A-42

context search See search

current line See dot (.)

cutting and pasting

- move command See m command
- procedures A-39

Ed (*continued*)

d command

- deletion A-9, A-42
- dot (.) setting A-36, A-42
- deletion See d command
- delimiter

- character choice A-26

description A-1

dollar sign (\$)

- last line notation A-6, A-9, A-28
- line end notation A-26, A-28
- metacharacter A-22, A-26
- multiple functions A-27

dot (.)

- current line notation A-6
- description A-8
- determination A-35
- search setting A-12, A-43
- substitution setting A-10
- symbol (.) A-8, A-24
- value determination A-9, A-43

duplication See t command

e command A-4, A-42

edit See e command

entry A-3

equals sign (=)

- dot value printing (.=) A-9, A-43
- last line value printing A-43

escape command (!) A-21, A-43

exclamation point (!), escape command A-21

f command A-5, A-42

file

- insertion into another file A-40
- writing out A-40

filename

- change A-5
- recovery A-5
- remembered filename printing A-5, A-42

folding A-20

g command

- a command combination A-19
- backslash () use A-18
- c command combination A-19

g command (*continued*)

- command combinations A-17, A-18
- dot (.) setting A-18
- i command combination A-19
- line number specifications A-18
- multiline construction A-19
- s command combination A-18, A-43
- search, command execution A-17, A-42
- substitution A-22
- trailing g A-22
- global command See g command
- global command See v command
- greater-than sign (>), tab notation A-19
- grep command A-22
- hyphen (-), character class A-31
- i command
 - backslash () characteristics A-26
 - dot (.) setting A-15, A-36, A-42
 - global combination A-19
 - input termination A-24
 - insertion A-15, A-42
- in-line input documents 7-47
- input
 - termination A-3, A-15, A-24
- insert command See i command
- INTERRUPT key
 - command execution effect A-39
 - dot (.) setting A-39
 - print stopping A-6
- introduction A-1
- invocation A-3
- j command, line joining A-34
- k command, line marking A-20
- k command
 - folding A-20
 - line listing A-19, A-43
 - nondisplay character printing A-19
 - number string A-20
 - s command combination A-23
- less-than sign (<)
 - backspace notation A-19

Ed (*continued*)

- line beginning
 - character deletion A-31
 - notation A-28
- line end
 - notation A-26
- line number
 - 0 as line number A-39
 - combinations A-6
 - summary A-42
- line
 - beginning See line beginning
 - break See splitting
 - folding A-20
 - joining A-34
 - marking A-20
 - moving See m command
 - number See line number
 - rearrangement A-34
 - splitting A-33
 - writing out A-41
- list See l command
- m command
 - dot (.) setting A-16, A-43
 - line moving A-15, A-43
 - warning A-16
- mail system See Mail
- marking See k command
- metacharacter
 - ampersand (&) A-32
 - asterisk (*) A-22, A-28
 - backslash () A-22, A-25
 - brackets ([]) A-22, A-31
 - caret (^) A-22, A-28
 - character class A-31
 - combination A-28
 - dollar sign (\$) A-22, A-26
 - escape A-26, A-33
 - period (.) A-22, A-23
 - search A-31
 - slash (/) A-22
 - star (*) A-22
- minus sign (-), address arithmetic A-6
- move
 - command See m command

move (*continued*)

- line marking A-20

- multicommand line restrictions A-11

- new line

- substitution A-33

- nondisplay character printing A-19

- p command

- dot (.) setting A-39

- multicommand line A-11

- printing A-5, A-43

- s command combination A-23

- pattern search See search

- period (.)

- a command input termination A-3, A-24

- c command input termination A-15

- character substitution A-23

- dot symbol See Dot (.)

- i command input termination A-24

- literal A-25

- metacharacter A-22, A-23

- s command, effect A-23

- script problems A-42

- search problems A-22

- troff command prefix A-17

- plus sign (+), address arithmetic A-6

- print

- command See p command

- line folding A-20

- RETURN key effect A-9

- stopping A-6

- q command

- abortion use A-43

- quit session A-4, A-43

- w command combination A-43

- question mark (?)

- search error message (?) A-12

- search repetition (??) A-14

- search, reverse direction (??) A-12, A-44

- write warning A-4

- quit See q command

- quotation marks, single (')

- line marking A-20

- r command

r command (*continued*)

- dot (.) setting A-37, A-43

- file insertion A-40

- positioning without address A-40

- read file A-5, A-43

- reading See r command

- regular expression

- description A-22

- metacharacter list A-22

- RETURN key, printing A-43

- s command

- ampersand (&) A-32

- character match A-23

- description, use A-9, A-43

- dot (.) setting A-10, A-36, A-43

- g command combination A-18, A-43

- l command combination A-23

- line number A-23

- new line A-33

- p command combination A-23

- search combination A-12

- text removal A-11

- trailing g A-22

- undoing A-20

- v command combination A-18

- script A-41

- search

- dot (.) setting A-43

- error message (?) A-12

- forward search (/ /) A-12, A-43

- global search See g command

- global search See v command

- metacharacter problems A-22

- next occurrence description A-12

- procedure A-11

- repetition (/ /), (??) A-14

- reverse direction (??) A-12

- separator A-38

- substitution combination A-12

- sed command A-22

- semicolon (;)

- dot (.) setting A-38

- search separator A-38

- shell

- shell (*continued*)
 - escape See escape command (!)
- s_oash (/)
 - delimiter A-26
 - literal A-26
 - metacharacter A-22
 - search forward (/ /) A-12, A-43
 - search repetition (/ /) A-14
- s_opecial character See metacharacter
- s_oelling correction See s command
- s_oar (*), metacharacter A-22
- substitution
 - command See s command
- t command
 - dot (.) setting A-43
 - transfer line A-21, A-43
- tab printing A-19
- t_ol command A-40
- termination See q command
- text
 - removal See s command
 - saving A-4
- t_oransfer See t command
- t_off command printing A-17
- typing error correction See s command
- u command
 - undo A-20, A-43
- u_ondo See u command
- v command
 - a command combination A-19
 - backslash () use A-18
 - c command combination A-19
 - command combinations A-17, A-18, A-19
 - dot (.) setting A-18
 - global search, substitute A-17, A-43
 - i command combination A-19
 - line number specifications A-18
 - s command combination A-18
- w command
 - description, use A-3
 - dot (.) setting A-37, A-43
 - e command combination A-42
 - w command (*continued*)
 - file write out A-40
 - frequent use advantages A-37
 - line write out A-41
 - write out A-3, A-4, A-40, A-43
 - write out
 - command See w command
 - warning A-4
- EDFIND shell procedure 7-47
- ~ editor escape See Mail
- Editor See Ed
- EDITOR string, mail 6-25, 6-37
- EDLAST shell procedure 7-47
- egrep See grep command
- elif clause See if command
- else clause See if command
- Else-part grammar 7-53
- Empty grammar 7-53
- ENTER key See RETURN key
- Equal sign (=)
 - BC
 - assignment operator symbol 8-5
 - relational operator 8-10, 8-18
 - ed use See Ed
 - mail, message number printing 6-14, 6-30
 - variable
 - conditional substitution 7-39
 - string value assignment 7-11
- errdirect file 7-28
- Error output redirection 7-38
- Escape string, mail 6-26, 6-37
- /etc/termcap file 4-4
- eval command
 - command line rescan 7-19
 - shell built-in command 7-41
- Ex, ed similarity A-1
- Exclamation point (!)
 - BC, relational operator 8-10, 8-18
 - ed use See Ed
 - mail
 - shell command execution 6-19, 6-23, 6-30
 - unary negation operator 7-35
 - vi See Vi

User's Guide

exec arg command 7-31
exec command 7-41
Exit code See \$? variable
exit command
 shell built-in command 7-41
 shell exit 7-25
 special shell command 7-30
Exit status
 \$? variable 7-14
 case command 7-23
 cd arg command 7-30
 colon command (:) 7-30
 command grouping 7-27
 false command 7-37
 if command 7-22
 read command 7-31
 true command 7-37
 until command 7-23
 wait command 7-32
 while command 7-23
export command
 shell built-in command 7-41
 variable
 example 7-13
 listing 7-16
 setting 7-16
expr command 7-36

F command, mail 6-11, 6-17, 6-30
f command
 ed use See Ed
 mail 6-10, 6-11, 6-17, 6-30
-f option, mail 6-9, 6-28
false command 7-37
fgrep See grep command
fi command
 if command end 7-21
 mail 6-30
File descriptor
 description, use 7-5
 redirection 7-7, 7-38
File permission
 changing 4-16
File permissions, listing 4-11

File system
 defined 3-3
 diagram 3-3
 organization 3-3
File
 access
 control 3-2
 last access time 3-1
 permission See Permission
 addition See creation
 alphabetizing See sort
 appending 4-7
 attributes 3-1
 binary file 3-1
 combining 4-6
 composition 3-1
 copying 4-7
 creating 4-4
 with vi 5-2
 creation
 MKFILES shell procedure 7-49
 permission See Permission
 time 3-1
 write permission control 3-2
 defined 3-1
 deleting 4-8
 deletion
 write permission control 3-2
 descriptor See File descriptor
 directory See Directory
 displaying 4-5, 4-6
 editing See Vi
 filename See Filename
 grammar 7-52
 inode number See Inode number
 linking 4-9
 listing 3-2
 mail system files See Mail
 manipulation 4-4
 modification time 3-1
 moving 4-7
 name See Filename
 pathname, printing 4-13
 pathname required 3-3
 pattern search See Ed

File (*continued*)

- pattern search See *grep* command
- pattern search See *Pattern matching* facility
- permission See *Permission*
- permissions 4-14
- pipe interchange 7-46
- protection 3-2
- removal 4-8
- renaming 4-7
- scratch file directory 3-5
- shell procedure creation 7-32
- size in bytes 3-1
- sorting 4-19
- special file See *Special file*
- temporary file See *Temporary file*
- textual contents determination 7-50
- types designated 3-1
- variable file creation See *Variable*

Filename

- argument 7-3
- asterisk (*) wildcard 3-6
- characters use restrictions 3-4
- description 3-3
- ed See *Ed*
- example designated 3-5
- long listing 4-11
- question mark (?) representation 3-6
- required 3-1, 3-3
- unique to directory 3-3, 3-4

Files

- comparing 4-18

Filter

- description 7-7
- order consideration 7-42

find command 4-8

Finding a file 4-8

finger command 4-24

Flag See *Option*

for command

- break command effect 7-25
- continue command effect 7-25
- description, use 7-23
- redirection 7-27
- shell built-in command 7-41

for loop, argument processing 7-17

Foreground process 4-22, 4-23

fork command 7-41

FSPLIT shell procedure 7-48

Full pathname See *Pathname*g command See *Ed*

G command

- vi See *Vi*

Global

- ed use See *Ed*
- variable check 7-33

goto command

- See *G* command 5-5

Greater-than sign (>)

- BC, relational operator 8-10, 8-18
- file combination 4-7
- output redirection 3-9
- PS2 variable default value 7-13
- redirection symbol 2-3, 7-54

grep command 4-19

- ed See *Ed*

Group permission See *Permission*

H command

- vi use See *Vi*

h command

- mail 6-8, 6-14, 6-30
- vi use See *Vi*

H flag, mail 6-15

head command 4-6

headers command See *Mail*~ headers escape See *Mail*ho command See *Mail*

Home directory 4-13

HOME variable

- conditional substitution 7-40
- description 7-12

-i option

- mail 6-27, 6-28, 6-37
- shell invocation 7-40

User's Guide

if command

- COPYTO shell procedure 7-45
- description, use 7-21
- exit status 7-22
- fi command required 7-22
- multiple testing procedure 7-21
- nesting 7-22
- redirection 7-27
- shell built-in command 7-41
- test command 7-34

IFS variable 7-12

ignore option See Mail

ignorecase option See Vi 5-34

In-line input document See Input

Inode number

- defined 3-2
- link See Link
- ls command 3-2
- required for file 3-1, 3-2

Input

- ed See Ed
- grammar 7-52
- in-line input
 - document 7-37
 - EDFIND shell procedure 7-47
- keyboard origin 3-8
- redirection See Redirection
- standard input file 7-5
- termination 4-2

Insert mode See Vi

Insertion See Ed

Internal field separator

- shell scanning sequence 7-18
- specificaiton by IFS variable 7-12

INTERRUPT key

- background process immunity 7-20
- BC 8-2
- command-line buffer cancellation 3-7
- ed use See Ed
- foreground process killing 4-23
- logging in, nonsense character
 - removal 2-2
- mail
 - askcc switch 6-24
 - message abortion 6-10, 6-25

INTERRUPT key (*continued*)

- program stopping 2-5

Interrupt

- handling methods 7-27
- key See INTERRUPT key

Invocation flag See Option

Item grammar 7-52

j command See Ed

j command

- vi use See Vi

k command See Ed

k command

- vi use See Vi
- k option, shell procedure 7-34
- Keyword parameter
 - description 7-16
 - k option effect 7-34

Kill character See CTRL-U

kill command 4-23

Killing a process 4-23

L command

- vi use See Vi

l command 4-11

- ed use See Ed
- mail 6-17, 6-30
- vi use See Vi

-l option

- function 3-8

lc command 4-10

- listing 2-3

Less-than sign (<)

- BC, relational operator 8-10, 8-18
- redirection symbol 7-54

Less-than symbol (<)

- input redirection 3-10

/lib directory

- contents 3-5

line command

- shell variable value assignment 7-9

- Line
 - beginning See Ed
 - counting See wc command
 - writing out See Ed
- linenumber option See Vi
- Line-oriented commands See Vi 5-12
- Lineprinter *See also* command
 - see
- Lineprinter *See also* options
 - see
- Lineprinter
 - status information 4-25
- Link
 - command See ln command
 - defined 3-2
 - description 4-9
 - long listing 4-11
- Linking files 4-9
- list command
 - mail 6-30
- list option See Vi
- LISTFIELDS shell procedure 7-48
- Listing directory contents 4-10
- Listing See l command
- Listing See lc command
- ln command 4-9
- Logging in 4-1
 - nonsense character removal 2-2
 - procedure 2-1
 - prompt character 2-2
 - resetting terminal characteristics 2-4
 - type-ahead not allowed 2-4
- Logging out
 - background process immunity 7-20
 - procedure 2-5, 4-2
 - shell termination 7-25
- Login directory
 - defined 7-12
 - new user 2-1
- Login message 2-2
- Login
 - procedure 4-1
- Looping
 - break command 7-25
 - continue command 7-25
- Looping (*continued*)
 - control 7-25
 - expr command 7-37
 - false command 7-37
 - for command 7-23
 - iteration counting procedure 7-37
 - time consumption 7-41
 - true command 7-37
 - unconditional loop implementation 7-37
 - until command 7-23
 - while command 7-23
 - while loop 7-45
- lp command
 - mail
 - m option 6-28
- Lp 4-27
- lp 4-26, 4-27, 4-28
- lpr command
 - mail
 - message printing 6-17, 6-30
- lpr *See also* command
 - see
- lpstat command 4-25
- lpstat 4-28
- ls command
 - echo * use in lieu of 7-36
 - function 3-2
 - inode number use 3-2
- m command
 - ed See Ed
 - mail 6-17, 6-30
- M flag See Mail
- m option, mail 6-28
- magic option See Vi
- mail command See Mail
- MAIL variable 7-12
- Mail
 - ? command See help command (?)
 - ` : See command escape (` :)
 - ` ? See help escape (` ?)
 - ` ! See shell escape (` !)
 - ` | See shell escape (` |)

User's Guide

Mail (*continued*)

`~` See tilde quote escape (`~`)

a command See alias

Alias 6-30

alias

a command 6-11, 6-18, 6-30

display 6-11

network mail 6-12

personal 6-11, 6-24

R command 6-11

system-wide 6-24

askcc option 6-24, 6-37

asksubject option 6-24, 6-37

asterisk (*)

character matching 6-7

message saved, header notation 6-14, 6-16

at sign (@), ignore switch echo 6-27, 6-37

autobox option

description, use 6-26, 6-37

effect 6-15

H flag 6-15

ho command 6-16

autoprint option 6-24, 6-37

`~b` escape 6-21

`-b` option 6-27

BACKSPACE key 6-6, 6-10

`~bcc` escape 6-35

Bcc field See blind carbon copy field

blind carbon copy field

description 6-5

editing 6-21, 6-22

escape See `~bcc` escape

box See Mailbox

`~c` escape 6-21

`-c` option 6-27

carbon copy field

blind See blind carbon copy field

description 6-5

display 6-4

editing 6-21

escape See `~c` escape

escape See `~cc` escape

option See askcc option

carbon copy field (*continued*)

R command effect 6-11

caret (^), first message specification 6-6, 6-13, 6-30

`~cc` escape 6-35

cc field See carbon copy field

cd command 6-19, 6-30

chron option 6-25, 6-37

colon (:)

escape See command escape (`~;`)

command escape (`~;`) 6-23, 6-35

command line options 6-27

command mode

description, use 6-6

help command 6-12

options setting 6-12

command

See also Specific Command

descriptions 6-12

escape See command escape (`~;`)

invocation 6-12

mail command See mail command

summary 6-30

syntax 6-8

compose escape (`~!`) 6-35

compose escape (`~)` 6-35

compose escapes

See also Specific Escape

compose mode exit 6-6

edit mode entry 6-6

heading escapes 6-20

listing 6-2

m command 6-17

reply 6-17

summary 6-34

compose mode

compose escapes See compose escapes

description, use 6-6

edit mode entry 6-6

entry from command mode 6-16

entry from shell 6-10

tilde escapes See compose escapes

concepts 6-4

CTRL-D

CTRL-D (*continued*)

message reply 6-11, 6-17

message sending 6-9

CTRL-H, backspace 6-6**CTRL-U**, line killing 6-6, 6-10

d command 4-29, 6-4, 6-7, 6-10, 6-15, 6-30

~d escape 6-22

~dead escape 6-36

dead.letter file

escape See ~d escape

nosave switch effect 6-25

undelivered message receipt 6-9

deletion See message

distribution list creation 6-11

dollar sign (\$), final message

specification 6-6, 6-13, 6-30

dot (.), current message specification 6-6, 6-13

dot option 6-25, 6-37

dp command 6-15, 6-30

e command 6-18, 6-30

~e escape 6-20

echo command 6-30

editor escape See ~e escape

editor escape See ~v escape

~editor escape 6-36

EDITOR string 6-25, 6-37

entry 6-9

equal sign (=), message number

printing 6-14, 6-30

escape string 6-26, 6-37

exclamation point (!)

shell command execution 6-19, 6-23, 6-30

exit

q command 4-29, 6-4, 6-9, 6-15, 6-30

x command 6-30

F command 6-11, 6-17, 6-30

f command 6-10, 6-11, 6-17, 6-30

-f option 6-9, 6-28

fi command 6-30

file switch See -f option

files designated 6-29

Mail (*continued*)

forwarding

procedure See F command

procedure See f command

h command 6-8, 6-14, 6-30

~h escape 6-21

H flag, message saving 6-15

header

characteristics 6-14

command See h command

defined 6-7

display 6-3, 6-7, 6-9

listing 6-30

windows 6-8

~headers escape 6-36

heading

compose escapes 6-20

composition 6-5

help command (?) 6-3, 6-12

help escape (~ ?) 6-20, 6-35

ho command

description 6-16

H flag 6-15

message saving 6-30

hold command See ho command

-i option 6-27, 6-28, 6-37

ignore switch See -i option

INTERRUPT key

message abortion 6-10, 6-25

recipient list 6-24

introduction 6-1

l command 6-17, 6-30

line killing 6-6, 6-10

list command 6-30

lp command

-m option 6-28

lpr command

message printing 6-17, 6-30

m command 6-17, 6-30

~M escape 6-22

~m escape 6-22

M flag, message saving 6-15

-m option 6-28

mail command

command mode entry 6-6, 6-9

mail command (*continued*)

- compose mode entry 6-10
- help 6-3
- message reading 6-3, 6-10
- message sending 6-1, 6-30
- mail escapes See ~M escape
- mail escapes See ~m escape
- mailbox See Mailbox
- .mailrc file
 - alias contents 6-18
 - distribution list creation 6-11
 - example 6-24
 - options setting 6-12
 - set command 6-18
 - unset command 6-18
- mb command 6-16, 6-30
- mbx command See mb command
- mchcron option 6-37
- ~Message escape 6-36
- ~message escape 6-36
- message list
 - composition 6-6
 - full message list description 6-8
- message number
 - command 6-14, 6-30
 - message printing 6-10
 - printing 6-14, 6-30
 - types 6-6
- message
 - abortion 6-10, 6-25
 - advancement 6-10, 6-30
 - body 6-5
 - composition 6-5
 - deletion undoing 6-15
 - deletion 4-29, 6-4, 6-7, 6-15, 6-30
 - description 6-5
 - display 4-29
 - editing 6-10, 6-18, 6-19, 6-28, 6-30
 - file inclusion 6-22
 - forwarding See forwarding
 - header See header
 - heading See heading
 - insertion into new message 6-22
 - list See message list
 - listing 6-3

message (*continued*)

- number See message number
- printing See printing
- range description 6-7
- reading into file 6-9
- reading 6-3, 6-10
- reply See reply
- saving See saving
- sending See sending
- size 6-19, 6-30
- undeletion 6-10
- metacharacters 6-6, 6-13
- metoo option 6-25, 6-37
- minus sign (-), message advancement 6-30
- network mail 6-12
- nosave option 6-25, 6-38
- number command See message number
- options
 - See also Specific Option
 - command line options 6-27
 - setting 6-12
 - summary 6-37
 - switch option setting 6-18
- p command
 - message printing 6-3, 6-7, 6-12, 6-30
 - syntax 6-8
- ~p escape 6-20
- page option 6-26
- period (.), dot use See dot (.)
- plus sign (+), message advancement 6-30
- ~print escape 6-36
- printing
 - command See lpr command
 - command See p command
 - escape See ~p escape
 - lineprinter See lpr command
 - procedure 6-7
- programs designated 6-29
- prompt 4-29, 6-3
- q command
 - exit 4-29, 6-4, 6-9, 6-15, 6-30

- q command (*continued*)
 - message abortion 6-25
- question mark (?)
 - command summary printing 6-30
 - help command 6-12
- quiet option 6-25, 6-38
- ~quit escape 6-36
- R command
 - alias effect 6-11
 - compose mode entry 6-10
 - message reply 6-11, 6-17, 6-30
- r command
 - compose mode entry 6-10
 - message reply 6-10, 6-11, 6-17, 6-30
- ~r escape 6-22
- R option 6-27
- r option 6-27
- read escape See ~d escape
- read escape See ~r escape
- ~read escape 6-36
- reading 4-29
- recipient list, name addition 6-20
- record string 6-26, 6-38
- reminder service 4-31, 6-28
- Reply command See R command
- reply command See r command
- return receipt request field 6-5
- s command
 - flag 6-14
 - message saving 6-16, 6-30
 - system mailbox, message deletion 6-15
- ~s escape 6-21
- s option 6-27
- saving
 - asterisk (*) notation 6-16
 - automatic 6-15
 - command See s command
 - flag 6-14
 - ho command 6-30
 - M flag 6-15
 - message display 6-4
 - s command 6-16, 6-30
 - system mailbox 6-9
- saving (*continued*)
 - w command 6-16, 6-30
- se command See set command
- sending 4-29
 - cancellation impossible 6-3
 - multiple recipients 6-9
 - network mail 6-12
 - procedure 6-9
 - to self 6-1
- session abortion 6-10
- set command
 - description, use 6-18, 6-30
 - option control 6-37
- set options defined 6-24
- sh command 6-19, 6-30
- shell commands 6-19
- shell escape (~!) 6-23
- shell escape (~) 6-23
- SHELL string 6-26, 6-38
- si command 6-19, 6-30
- so command 6-19, 6-30
- source command See so command
- special characters See metacharacters
- startup file 6-24
- string option
 - setting 6-18
 - summary 6-37
- subject escape See ~s escape
- ~subject escape 6-36
- subject field 6-4, 6-5
- subject switch See asksubject option
- subject switch See -s option
- switch See Option
- system composition 6-29
- system mailbox, message retention 6-9
- t command
 - message top printing 6-14, 6-30
 - toplines option 6-14
- ~t escape 6-20
- tilde escapes See compose escapes
- tilde quote escape (~") 6-23, 6-35
- ~to escape 6-36
- to field
 - mandatory 6-5

- to field (*continued*)
 - R command effect 6-11
 - toplines option 6-38
 - toplines string 6-27
 - u command 6-7, 6-10, 6-15, 6-30
 - u option 6-27
 - undeletion See u command
 - unset command
 - description, use 6-18, 6-30
 - option control 6-37
 - v command 6-6, 6-19, 6-30
 - ~v escape 6-20
 - variable See MAIL variable
 - vertical bar (|) escape See shell escape (`|`)
 - ~visual escape 6-36
 - VISUAL string 6-26, 6-38
 - w command
 - message write out 6-16, 6-30
 - system mailbox, message deletion 6-15
 - ~w escape 6-22
 - write escape See ~w escape
 - ~write escape 6-36
 - write out See w command
 - x command
 - exit 6-30
 - session abortion 6-10
 - you have mail message 2-2
- Mailbox
 - command 6-16
 - reading in 6-9
 - system mailbox 6-5
 - user mailbox
 - filename 6-5
 - message saving notation 6-15
- Make directory See mkdir command
- Marking See Ed
- mb command See Mail
- mbx command See Mail
- mchcron option
 - mail 6-37
- mesg option See Vi
- ~Message escape See Mail
- ~message escape See Mail
- Metacharacter
 - asterisk (*) 7-54
 - brackets ([]) 7-54
 - directory name use avoidance 7-4
 - escape 7-4
 - list designated 7-54
 - mail 6-6, 6-13
 - question mark (?) 7-54
 - redirection restriction 7-6
- metoo option See Mail
- Minus sign (-)
 - BC
 - subtraction operator symbol 8-4
- Minus sign (-)
 - BC
 - unary operator symbol 8-4, 8-16
 - mail, message advancement 6-30
 - redirection effect 7-37
- Minus sign (-)
 - subtraction operator symbol 8-4
- Minus sign (-)
 - variable conditional substitution 7-39
- mkdir command 4-12
- MKFILES shell procedure 7-49
- more command 4-5
- Move See mv command
- Multiple way branch See case command
- Multiplication See BC
- mv command 4-7
 - - directory moving 4-12
- n command See Vi
- n option
 - echo command 7-36
 - shell procedure 7-34
- Name grammar 7-53
- Name special file 4-14
- Named pipe 4-14
- newgrp command
 - description 7-31
 - shell built-in command 7-41
 - special shell command 7-31
- Newline substitution See Ed
- next command See Vi 5-41

nohup command 7-20
 nosave option See Mail
 nu command See Vi 5-22
 Null command See Colon command (:)
 NULL shell procedure 7-49
 Number sign (#), comment symbol 7-54

-o operator 7-35
 (o), write command message end 4-30
 (oo), write command message end 4-30

Operator See BC

Option

See also Specific Option
 configuration 3-8
 DRAFT shell procedure 7-47
 grouping 3-8
 invocation flags 7-40
 mail options See Mail
 position 3-8
 tracing, \$- variable 7-14

Options

terminal 4-4
 vi options See Vi

Ordinary file See File

Or-if operator (||)

command list 7-19
 description, use 7-20
 designated 7-54

Output

append symbol (>>) 7-6, 7-54
 appending
 procedure 3-9
 symbol (>>) 3-9
 control 4-4
 creation symbol (>) 7-54
 diagnostic output file 7-5
 error redirection 7-38
 grammar 7-52
 redirection See Redirection
 redirection 2-3, 4-7
 resumption 4-4
 standard error file See diagnostic
 output file
 standard output file 7-5

Output (*continued*)

terminal screen destination 3-8
 to file 2-3

p command

ed use See Ed
 mail

message printing 6-3, 6-7, 6-12, 6-30

syntax 6-8

page option See Mail

Parent directory

description 3-5
 shorthand name 3-5

Parentheses (())

BC

expression enclosure 8-15
 function identifier argument
 enclosure 8-14

command grouping 7-26, 7-41, 7-54
 pipeline, command list enclosure 7-21
 test command operator 7-35

passwd command 4-2

Password

changing 4-2
 logging in 2-1

PATH variable

conditional substitution 7-40
 description 7-12
 directory search
 effect 7-42
 sequence change 7-3

Pathname

absolute pathname
 example 3-5
 required 3-3
 slash (/) significance 3-4
 unique to system 3-3

defined 3-4

full pathname See absolute pathname

relative pathname

defined 3-4
 example designated 3-5
 structure 3-4

Pattern matching facility
cancellation 3-7
case command 7-22
description 3-5
expr command argument effect 7-36
grep command 4-19
limitations 7-3
redirection restriction 7-6
variable assignment, not applicable 7-11

Pattern
grammar 7-52
metacharacter 7-54

Percentage sign (%), BC modulo
operator symbol 8-4

Period (.)
ed use See Ed
filename use 3-4
pattern matching facility restrictions 7-4
vi See Vi
working directory change 4-13

Permission types 4-15

Permission
block special device notation 4-14
change 3-2
denial notation 4-15
directory permission
assignment 3-2
change 3-2, 4-16
combinations designated 4-15
file creation, deletion notation 4-15
file listing notation 4-15
notation 4-14
search notation 4-15
search permission 4-17
write permission 3-2
execute notation 4-15
file permission
change 3-2
denial notation 4-15
execute permission 4-15
file creation, deletion notation 4-15
file listing notation 4-15
file protection 3-2

file permission (*continued*)
notation 4-14
read notation 4-15
required 3-1
write notation 4-15

listing 4-14
notation 4-14
read notation 4-15
search notation 4-15
symbols designated 4-14
user class specification 4-17
write notation 4-15

PHONE shell procedure 7-50

PID

\$! variable 7-14
\$\$ variable 7-14
process identification number 4-23, 4-24

Pipe

compose escapes See Mail
file interchange 7-46
function 3-10
procedure 3-10
symbol (|) 3-10, 7-54

Pipeline

command list 7-21
defined 3-10, 7-19
description 7-7
DISTINCT1 shell procedure 7-46
filter 7-7
grammar 7-52
notation designated 7-7
procedure 7-7

Plus sign (+)

BC

addition operator symbol 8-4
unary operator symbol 8-16
mail, message advancement 6-10, 6-30
variable conditional substitution 7-39

Positional parameter

description 7-10
direct access 7-17
null value assignment 7-39
number yield, \$# variable 7-13

- Positional parameter (*continued*)
 - parameter substitution 7-11
 - positioning 7-11
 - prefix (\$) 7-11
 - setting 7-10
 - variable assignment statement
 - positioning 7-11
- ~ print escape See Mail
- Print working directory See pwd
 - command
- Printing *See also* files
 - see
- Printing
 - command See p command
 - ed See Ed
 - mail See Mail
- Process identification number See PID
- Process
 - background See Background process
 - defined 7-1
 - foreground See Foreground process
 - number See PID
 - status 4-24
- .profile file
 - description, use 7-15
 - PATH variable setting 7-13
 - variable export 7-13
- Program stopping 2-5
- Prompt character 2-2, 4-1
- ps command 4-23, 4-24
- PS1 variable 7-13
- PS2 variable 7-13
- pwd command 4-10, 4-13
- q command
 - mail
 - exit 4-29, 6-4, 6-9, 6-15, 6-30
 - message abortion 6-25
- Question mark (?)
 - directory name, use avoidance 7-4
 - ed use See Ed
 - filename, use avoidance 3-4
 - mail
 - command summary printing 6-30
 - mail (*continued*)
 - compose escape listing 6-2, 6-20
 - help command 6-3, 6-12
 - metacharacter 7-54
 - pattern-matching functions 3-6
 - single character representation 3-6
 - variable conditional substitution 7-39
- quiet option See Mail
- quit command
 - See also q command
 - BC exit 8-2, 8-3
- ~ quit escape See Mail
- QUIT key, background process
 - immunity 7-20
- Quit See q command
- Quotation marks, back (``)
 - command line substitution 7-9
 - command substitution 7-4, 7-9
 - quoting 7-54
- Quotation marks, double
 - c option 7-40
 - filename, use avoidance 3-4
 - metacharacter escape 7-4
 - quoting 7-54
 - test command 7-34
 - variable 7-11
- Quotation marks, single (')
 - filename, use avoidance 3-4
 - grep command 4-20
 - metacharacter escape 7-4
 - pattern matching cancellation 3-7
 - trap command 7-28
 - variable substitution inhibition 7-11
- Quoting
 - backslash () use 7-54
 - metacharacter escape 7-4
 - quotation marks, back (``) use 7-54
 - quotation marks, double use 7-54
- r character, read permission notation 4-15
- R command See Mail
- r command
 - ed use See Ed

r command (*continued*)

- mail use See Mail
- R option, recursive listing 4-11
- R option
 - mail 6-27
- r option 3-8
- r option
 - mail 6-27
- Read command
 - vi See Vi
- read command
 - exit status 7-31
 - shell built-in command 7-41
 - special shell command 7-31
- ~ read escape See Mail
- Read-ahead 2-4
- readonly command
 - description 7-31
 - shell built-in command 7-41
 - special shell command 7-31
- Record string See Mail
- Redirection
 - argument location 7-8
 - case command 7-27
 - cd arg command 7-31
 - control command 7-27
 - diagnostic output 7-6
 - file descriptor 7-38
 - for command 7-27
 - if command 7-27
 - input redirection
 - procedure 3-10
 - symbol (<) 3-10
 - minus sign (-) effect 7-37
 - output redirection 4-7
 - symbol (>) 3-9
 - pattern matching use restriction 7-6
 - simple command line, appearance 7-19
 - special character use restriction 7-6
 - special shell command, restriction 7-30
 - symbol (>) 2-3
 - symbol (<) 7-54
 - symbol (>) 7-54

Redirection (*continued*)

- until command 7-27
 - while command 7-27
- ### **Reference Manual**
- directory removal information 4-12
 - linking information 4-10
 - sort command information 4-19
 - stty information 4-4
- ### **Regular expressions** See Ed
- ### **Relative pathname** See Pathname
- ### **Reminder service**
- automatic 4-31
 - mail 6-28
- ### **Remove directory** See rmdir command
- ### **Remove** See rm command
- ### **Removing a directory** 4-12
- ### **Renaming a file** 4-7
- ### **Repeat command**
- see Vi 5-39
- ### **reply command** See Mail
- ### **Report option** See Vi
- ### **Request** 4-26
- ### **Reserved word listing** 7-55
- ### **Return code** See \$? variable
- ### **RETURN key**
- BC 8-2
 - command execution 2-2, 4-4
 - command-line buffer submittal 3-7
 - mail, message display 4-29
- ### **rm** command 2-4, 4-8
- ### **rmdir** command 4-12
-
- ### **s** command
- ed use See Ed
 - mail 6-14, 6-15, 6-16, 6-30
- ### **-s** option
- mail, subject specification 6-27
 - shell invocation 7-40
- ### **scale** command 8-7
- ### **Scale** See BC
- ### **Screen** See Scrolling screen
- ### **Screen** See Terminal screen
- ### **Screen-oriented commands** See Vi 5-12
- ### **Scripts** See Ed

- Scripts See Shell
- Scrolling commands
 - more 4-5
- Scrolling, control 4-4
- Scrolling screen
 - stopping 4-4
- se command See set command
- Search permission See Permission
- Search See Ed
- Searching for a file 4-8
- Searching See / command
- Searching See Vi
- Searching
 - vi procedure See Vi
- sed command See Ed
- Semaphore 4-14
- Semicolon (;)
 - BC, statement separation 8-3, 8-19
 - case command break 7-22
 - case delimiter symbol 7-54
 - command list 7-19
 - command separation 3-7
 - command separator symbol 7-54
 - ed use See Ed
- set all See Vi
- set command
 - mail
 - description, use 6-18, 6-30
 - option control 6-37
 - name-value pair listing 7-16
 - positional parameters setting 7-10
 - shell built-in command 7-41
 - shell flag setting 7-15
 - special shell command 7-30
- sh command
 - description 7-1
 - mail 6-19, 6-30
 - shell invocation 7-17
- Shell command
 - executing while in vi 5-14
- Shell language defined 2-5
- SHELL string 6-26, 6-38
- Shell
 - argument passing 7-17
 - command interpretation 3-7
- Shell (*continued*)
 - command
 - search procedure 7-2
 - special command See special command
 - compose escapes See Mail
 - conditional capability 7-21
 - creation
 - procedure 7-1
 - description 7-1
 - e option 7-33
 - echo command 4-19
 - entry, mail mode source 6-19
 - escape
 - ed procedure See Ed
 - mail procedure See Mail
 - execution
 - flag See option
 - sequence 7-18
 - termination 7-25
 - exit
 - e option 7-33
 - mail mode return 6-19
 - procedure 7-25
 - t option 7-34
 - function 7-1
 - grammar 7-52
 - in-line input document handling 7-37
 - interactive 7-40
 - interruption procedure 7-27
 - invocation
 - option 7-40
 - procedure 7-17
 - k option 7-34
 - mail
 - invocation 6-6
 - shell commands 6-19
 - n option 7-34
 - option
 - See also Specific Option
 - designated, use 7-33
 - setting 7-15
 - positional parameter See Positional parameter
 - procedure

procedure (*continued*)

- See also Specific Shell Procedure
- advantages over C programs 7-33
- byte access reduction consideration 7-42
- creation 7-32
- description 7-2
- directory 7-33
- efficiency analysis 7-42
- efficiency awareness 7-41
- examples designated 7-44
- filter order consideration 7-42
- option See option
- scripts designation 7-44
- time command 7-41
- writing strategies 7-40
- redirection ability 7-6
- scripts See procedure
- special command
 - See also Specific Special Command
 - designated 7-30
 - redirection restriction 7-30
- special shell variable 7-18
- state 7-14
- string See SHELL string
- t option 7-34
- TERM variable See TERM variable
- u option 7-33
- v option 7-15
- variable See Variable
- x option 7-15
- shift command
 - argument processing 7-17
 - shell built-in command 7-41
- si command See Mail
- Simple command See Command
- Single quotation marks See Quotation marks, single (')
- Slash (/)
 - absolute pathname significance 3-4
 - BC, division operator symbol 8-4
 - command prepending suppression 7-2
 - ed use See Ed
 - pathname significance 3-4

Slash (/) (*continued*)

- search command See Vi
- so command See Mail
- sort command 4-19
- Special character See Metacharacter
- Special characters
 - pattern matching 3-5
- Special file
 - description 3-2
- Sshared data file 4-14
- Standard error file See Output
- Standard error output See Error output
- Standard input file See Input
- Standard output file See Output
- Status
 - command See ps command
 - information procedures 4-24
- String option See Mail
- String variable 7-10
- String
 - searching for See Search
- stty command 4-4
 - terminal setting 2-4
- Subdirectory 4-13
- ~ subject escape See Mail
- Subshell, directory change 7-15
- Substitution command See s command
- Subtraction See BC 8-4
- Switch See Option
- Switch
 - defined 3-8
- System
 - basic concepts 3-1
 - characteristics 1-1
 - composition 1-1
 - mailbox See Mailbox
- t command
 - ed use See Ed
 - mail 6-14, 6-30
- t option, shell procedure 7-34
- Table command See Ed
- Tabs
 - ed See Ed

- tail command 4-6
- tbl command See Ed
- Temporary file
 - directory (/tmp) 4-23
 - kill command warning 4-23
 - trap command, removal 7-28
 - use recommendation 7-14
- term option See Vi
- TERM variable, changing 4-3
- Terminal screen
 - output See Output
 - scrolling screen See Scrolling screen
- Terminal
 - changing 4-3
 - name designation 2-2
 - options setting 4-4
 - resetting 2-4
 - strange behavior remedy 2-4
 - writing to See write command
- Terminals
 - supported 4-3
- terse option See Vi
- test command
 - argument 7-35
 - brackets ([]) use in lieu of 7-34
 - description, use 7-34
 - operators 7-35
 - options 7-34
 - shell built-in command 7-41
- Text editor
 - ed See Ed
 - ex See Ex
 - purpose 2-5
 - vi See Vi
- TEXTFILE shell procedure 7-50
- then clause See if command
- Tilde escape See Mail
- time command 7-41
- /tmp directory 4-23
 - contents 3-5
- ~ to escape See Mail
- Toplines option See Mail
- Toplines string See Mail
- Transfer command See t command
- trap command
 - (continued)
 - description, use 7-27
 - implementation method 7-29
 - multiple traps 7-29
 - special shell command 7-30
 - temporary file removal 7-28
- troff See Ed
- true command 7-37
- tty, terminal system name 2-2
- Type-ahead 2-4, 4-4
- Typing error correction 2-4
- u command See vi 5-39
- u command
 - ed use See Ed
 - mail 6-7, 6-15, 6-30
 - vi See Vi
- u option, shell procedure 7-33
- u option
 - mail 6-27
- ugo, permission classification 4-17
- umask command
 - description 7-31
 - directory permission change 3-2
 - shell built-in command 7-41
 - special shell command 7-31
- Undo command See u command
- Undo command See Vi
- undo command See Vi 5-39
- unset command See Mail
- until command
 - continue command effect 7-25
 - description, use 7-23
 - exit status 7-23
 - redirection 7-27
 - shell built-in command 7-41
- User classes 4-16
- User
 - classification 4-16
 - mailbox See Mailbox
 - permission See Permission
- /usr directory
 - contents 3-4
- /usr/bin directory

User's Guide

/usr/bin directory (*continued*)

- /bin duplicate determination 7-44
- contents 3-4

v command

- ed use See Ed
- mail 6-6, 6-19, 6-30

- v option, input line printing 7-15

Value See \$? variable

Variable

- \$# variable 7-13
- \$! variable 7-14
- \$\$ variable 7-14
- \$_ variable 7-14
- \$? variable 7-14
- assignment
 - line command 7-9
 - string value 7-10
- BC variable See BC
- command environment composition 7-16
- conditional substitution 7-38
- description 7-10
- double quotation marks () 7-11
- enclosure 7-11
- execution sequence 7-11
- expansion 7-4
- export 7-13
- expr command 7-36
- file creation 7-27
- global check 7-33
- HOME See HOME variable
- IFS See IFS variable
- keyword parameter 7-16
- listing procedure 7-16
- MAIL See MAIL variable
- name defined 7-11
- null value assignment procedure 7-39
- PATH See PATH variable
- positional parameter See Positional parameter
- prefix (\$) 7-11
- PS1 See PS1 variable
- PS2 See PS2 variable

Variable (*continued*)

- set variable defined 7-38
- special variable 7-13
- string value assignment 7-11
- substitution
 - double quotation marks 7-11
 - notation 7-54
 - redirection argument 7-6
 - single quotation marks (') 7-11
 - space interpretation 7-11
 - u option effect 7-33
- test command 7-34
- types designated 7-12

Vertical bar (|)

- mail escape 6-23
- or-if operator symbol (||) 7-19
- pipe symbol 3-10

Vertical bar (|)

- pipeline notation 7-7

Vi, mail

- compose escape, ~ v 6-36
- editing 6-19
- entry from command mode 6-6
- entry from compose mode 6-6
- VISUAL string 6-38

Vi

- / command
 - searching 5-9
- . command
 - See dot (.) command
- . command See dot (.) command
- 0 command
 - cursor movement 5-5
- args command 5-41
- b command, cursor movement 5-5
- breaking lines 5-25
- buffers
 - delete See delete buffer
 - delete 5-32
 - naming 5-22
 - selecting 5-22
- C command 5-29
- C shell
 - prompt 5-47
 - TERM variable 5-47

C shell (*continued*)

- terminal type setting 5-47
- canceling changes 5-40
- caret (^), pattern matching 5-37, 5-38
- cc command 5-29
- co (copy) command 5-22
- colon (:)
 - line-oriented command, use 5-12
 - status line prompt 5-12
- command mode
 - cursor movement 5-5
- command
 - line-oriented See line-oriented commands 5-12
 - repeating, dot (.) use 5-6
 - screen-oriented See screen-oriented commands 5-12
- control characters, inserting 5-25
- copying lines 5-22
- correcting mistakes 5-20
- crash, recovery 5-45
- CTRL-B
 - scrolling 5-6
- CTRL-D
 - scrolling 5-5
 - subshell exit 5-45
- CTRL-F
 - scrolling 5-6
- CTRL-G
 - file status information 5-10, 5-44
- CTRL-J, inserting 5-25
- CTRL-L
 - screen redraw 5-45
- CTRL-Q, inserting 5-25
- CTRL-S, inserting 5-25
- CTRL-U
 - deleting an insertion 5-27
 - scrolling 5-5
- CTRL-V, use 5-25
- current line
 - deleting 5-6, 5-26
 - designated 5-2
 - line containing cursor 5-3
 - number, finding out 5-22
- cursor movement

cursor movement (*continued*)

- \$ key 5-18
- + key 5-18
- B 5-17
- backward 5-18
- b 5-17
- BKSP 5-16
- by character 5-16
- by lines 5-18
- by words 5-17
- CTRL-N 5-18
- CTRL-P 5-18
- down 5-5, 5-16
- E 5-17
- e 5-17
- F 5-17
- f 5-17
- forward 5-18
- H 5-19
- h 5-16
- j 5-16, 5-18
- keys 5-5
- k 5-16, 5-18
- L 5-19
- left 5-5, 5-16, 5-17
- line beginning 5-5
- line end 5-5
- LINEFEED key 5-18
- l 5-16
- lower left screen 5-5
- M 5-19
- RETURN key 5-18
- right 5-5, 5-16, 5-17
- screen 5-19
- scrolling See scrolling 5-5
- See also scrolling
- SPACEBAR 5-16
- T 5-17
- to end of file 5-5
- t 5-17
- upper left screen 5-5
- up 5-5, 5-16
- W 5-17
- word backward 5-5
- word forward 5-5

cursor movement (*continued*)

- w 5-17
- cursor movement
 - right 5-17
- cw command 5-28
- D command 5-6
- d\$ command 5-6
- d0 command 5-6
- date, finding out 5-14
- dd command 5-6
- delete buffer
 - use 5-32
- deleting text
 - by character 5-25
 - by line 5-26
 - by word 5-25
 - D 5-26
 - dd command 5-26
 - deleting an insertion 5-27
 - dw command 5-25
 - methods 5-6
 - repeating deletion 5-39
 - undoing deletion 5-4
 - undoing 5-39
 - X command 5-25
 - x command 5-25
- demonstration 5-1
- description 5-1
- dollar sign (\$)
 - cursor movement 5-5
 - pattern matching 5-37
- dollar sign(\$)
 - use in line address 5-26
- dot (.)command See . command 5-6
- dot See also dot (.) command
- dot, use in line address 5-26
- dw command 5-6
- editing several files
 - changing the order 5-41
- end-of-line
 - displaying 5-48
- entering
 - at a specified line 5-16
 - at a specified word 5-16
 - procedure 5-2

entering (*continued*)

- with filename 5-15
- with several filenames 5-40
- error messages
 - shortening 5-49
 - turning off 5-43
- ESCAPE, insert mode exit 5-45
- exclamation point (!)
 - shell escape 5-14
- exiting
 - saving changes 5-39
 - saving file 5-13
 - temporarily 5-14, 5-43
 - without saving changes 5-39
 - :x command 5-39
 - :x 5-15
 - ZZ command 5-39
- .exrc file 5-51
- file
 - creating 5-2
 - saving 5-15
 - status information display 5-10
 - status information procedure 5-10
- filename
 - finding out 5-44
 - planning 5-40
- G command
 - cursor movement 5-5
- goto command See G command
- H command
 - cursor movement 5-5
- h command
 - cursor movement 5-5
- i command
 - inserting text 5-3
- ignorecase option 5-34, 5-48
- insert command See i command
- insert mode
 - entering 5-3
- inserting text from another file 5-13
- inserting text
 - control characters 5-25
 - from other files 5-21
 - repeating insert 5-21, 5-39
 - undoing insertion 5-4

- inserting text (*continued*)
 - undoing insert 5-45
 - undoing 5-39
- invoking See entering
- j command
 - cursor movement 5-5
- joining lines 5-25
- k command
 - cursor movement 5-5
- L command
 - cursor movement 5-5
- l command
 - cursor movement 5-5
- leaving See exiting
- line addressing
 - dollar sign 5-26
 - dot(.) 5-26
 - procedure 5-26
- line numbers, displaying
 - linenumber option 5-14, 5-22, 5-49
 - nu command 5-22
 - :nu command 5-45
- linenumber option 5-49
- line-oriented command mode 5-44
- line-oriented commands
 - :args 5-41
 - colon (:) use 5-12
 - deleting text 5-26
 - :e 5-21
 - :e# 5-22
 - :e 5-42
 - :e# 5-42
 - entering 5-12
 - :f 5-44
 - :file 5-44
 - moving text 5-30
 - :n 5-41
 - :nu 5-22, 5-45
 - :q 5-40
 - :rew 5-42
 - :r 5-21
 - :s 5-29
 - status line, display 5-10
 - :wq 5-39
 - :w 5-21
- Vi (*continued*)
 - list option 5-48
 - .login file
 - terminal type setting use 5-47
 - magic option 5-38, 5-50
 - marking lines 5-21
 - mesg option 5-50
 - mistakes, correcting 5-20
 - mode, determining 5-45
 - mode
 - See also command mode 5-45
 - See also insert mode 5-45
 - See also line-oriented command mode 5-45
 - moving text 5-30
 - n command 5-10, 5-34
 - new line, opening 5-20
 - next command 5-41
 - opening a new line 5-20
 - options
 - displaying 5-48
 - ignorecase 5-34, 5-48
 - linenumber 5-22, 5-30, 5-49
 - list 5-15, 5-48
 - magic 5-38, 5-50
 - mesg 5-50
 - report 5-49
 - setting 5-46, 5-48
 - term 5-49
 - terse 5-49
 - warn 5-43, 5-50
 - wrapscreen 5-35, 5-50
 - overstrike commands 5-27
 - pattern matching
 - See also searching
 - beginning of line 5-37
 - caret (^) 5-38
 - character range 5-38
 - character, single 5-37
 - end of line 5-37
 - exceptions 5-38
 - generally 5-37
 - special characters 5-38
 - square brackets ([]) 5-38
 - period (.)

- period (.) (*continued*)
 - pattern matching 5-37
 - repeat command symbol 5-3
- problem solving 5-45
- .profile file
 - terminal type setting 5-47
- putting 5-22
- Q command, line-oriented command
 - mode 5-44
- quitting See exiting 5-40
- r comand 5-13
- r command 5-27
- read command 5-13
- redrawing the screen 5-45
- repeat command 5-39
- repeating a command 5-39
- replacing a line 5-29
- replacing a word 5-28, 5-29
- report option 5-49
- rew command 5-42
- S command 5-28
- s command 5-28
- saving a file 5-40
- screen, redrawing 5-45
- screen-oriented commands 5-12
- scrolling
 - backward 5-5, 5-6
 - down 5-5
 - forward 5-6
 - up 5-5
- searching and replacing
 - a word 5-35
 - c option 5-36
 - choosing replacement 5-36
 - command syntax 5-35
 - p option 5-36
 - printing replacement 5-36
- searching See / command
- searching
 - See also searching and replacing
 - backward 5-34
 - caret (^) use 5-37
 - caret 5-37
 - caret(^) 5-38
 - case significance 5-48
 - searching (*continued*)
 - case significance 5-34
 - dollar sign (\$) 5-37
 - forward 5-10, 5-34
 - next command 5-34
 - period (.) 5-37
 - procedure 5-9
 - repetition 5-10
 - special characters 5-34, 5-50
 - square brackets ([]) 5-38
 - status line, display 5-10
 - wrap 5-10, 5-35, 5-50
- set all, option list 5-15
- set command 5-46, 5-48
- setting options 5-48
- shell command, executing 5-14
- shell escape 5-43
- slash (/)
 - search command delimiter 5-9
- special characters
 - matching 5-38
 - searching for 5-34, 5-50
 - vi filenames 5-40
- status line
 - line-oriented command entry 5-12
 - location 5-10
 - prompt, colon (:) use 5-12
- string
 - pattern matching 5-38
 - searching for See searching
- subshell
 - exiting 5-45
- Substitute commands 5-28
- switching files 5-42
- system crash
 - file recovery 5-46
- tabs
 - displaying 5-48
- TERM variable 5-47
 - Bourne shell 5-47
 - Visual Shell 5-47
- termcap 5-47
- terminal type setting
 - Bourne shell 5-47
 - C shell 5-47

- terminal type setting (*continued*)
 - how 5-49
 - Visual Shell 5-47
- terse option 5-49
- text editor notation 2-5
- time, finding out 5-14
- u command 5-4, 5-39, 5-45
- undo command See u command
- undoing a command 5-39
- w command, cursor movement 5-5
- warn option 5-43, 5-50
- warnings, turning off 5-50
- word, deleting 5-6
- wrapsan option 5-35, 5-50
- write messages 5-50
- writing out a file
 - :wq command 5-39, 5-40
- x command 5-6
- :x command 5-39
- :x 5-15
- yanking lines 5-22, 5-24
- ZZ command 5-39
- visual command See Mail
- ~ visual escape See Mail
- Visual Shell
 - TERM variable 5-47
 - terminal type 5-47
- VISUAL string See Mail
- w character
 - directory permission notation 4-15
 - file permission, write notation 4-15
- w command See Vi
- w command
 - ed use See Ed
 - mail
 - message saving 6-16
 - message write out 6-30
 - system mailbox, message deletion 6-15
- wait command
 - description 7-32
 - exit status 7-32
 - shell built-in command 7-41
- wait command (*continued*)
 - special shell command 7-32
- warn option See Vi
- wc command 4-20
- while command
 - break command effect 7-25
 - continue command effect 7-25
 - description, use 7-23
 - exit status 7-23
 - loop 7-45
 - redirection 7-27
 - shell built-in command 7-41
 - test command 7-34
- who command 4-24
 - description 2-2
- Word
 - counting See wc command
 - grammar 7-53
- Working directory See Current directory
- wrapsan option See Vi
- write command 4-29
- ~ write escape See Mail
- WRITEMAIL shell procedure 7-51
- x character
 - directory permission search 4-15
 - file permission, execute notation 4-15
- x command See Vi
- x command
 - mail
 - exit 6-30
 - session abortion 6-10
 - vi use See Vi
 - x option, command printing 7-15
- ZZ command See Vi 5-39
- z
 - vi scroll 5-19

